Arabian Journal for Science and Engineering
https://doi.org/10.1007/s13369-026-11660-w

RESEARCH ARTICLE-SYSTEMS ENGINEERING

LUMA-YOLO: A Lightweight and Illumination-Adaptive Object Detector for Low-Light Autonomous Driving

Mingkun Feng[1] · **Chengjie Li**[1] · **Zhengxing Zhang**[2]

Received: 18 May 2026 / Accepted: 26 August 2026
© King Fahd University of Petroleum & Minerals 2026

Abstract
Low-light road object detection remains challenging for autonomous driving because uneven illumination, glare, and sensor noise weaken object contours and increase missed detections. This study proposes LUMA-YOLO, a lightweight illumination-adaptive detector based on YOLO26. A Lightweight Illumination-Adaptive Module (LIAM) is inserted before the backbone so that input calibration is optimized together with object detection. Average Pooling Convolution (AConv) and a Memory-Modulated Bottleneck (MMB) are used to suppress noise while retaining weak edge and texture cues. In the neck, C2PSA is removed, SPPF is replaced with SPPELAN, and the parameter-free SimAM module is introduced to strengthen multiscale feature responses without adding learnable parameters. On the ExDark, NOD, and TDND datasets, LUMA-YOLO achieves mAP@0.5 values of 0.709, 0.657, and 0.725, and mAP@0.5:0.95 values of 0.451, 0.434, and 0.461, respectively, with only 2.47 M parameters and 57.8 FPS on an RTX 5060 Ti GPU. Compared with recent YOLO-family detectors, adverse-condition detectors, and D-FINE-N, LUMA-YOLO gives the best overall balance of recall, F1-score, mAP, model size, and real-time speed; on the Jetson Orin Nano, it also retains the highest recall, F1-score, and mAP on the NOD dataset with an inference time of 42.87 ms. The code is available at https://github.com/Softnameli/YOLO.

Keywords Autonomous driving · Low-light object detection · Illumination adaptation · Lightweight detector · Real-time detection

1 Introduction

In recent years, with rapid advances in deep learning, numerous mainstream object detection models [1–4] have achieved excellent performance under well-lit conditions. However, in all-weather autonomous driving scenarios, complex and dynamic road environments impose far more stringent requirements on visual perception. Unlike the sufficient and uniform illumination available during the day, low-light road scenes are often affected by insufficient overall brightness and interference from oncoming headlights. Under such extreme high-contrast conditions, input images are susceptible to local glare and noise, while the contours of distant pedestrians and non-motorized vehicles are easily obscured by the background. Consequently, conventional object detection models often struggle to extract discriminative features in low-light road scenes, leading to frequent missed detections and false positives.

To improve road object detection accuracy under low-light conditions, recent studies have mainly followed two routes. The first is to introduce low-light image enhancement as a front-end preprocessing module before images are fed into the detection network. For example, Cai et al. [5] proposed Retinexformer, a Transformer-based enhancement model built on single-stage Retinex theory. By estimating illumination information, the model enhances low-light regions and serves as a front-end module to improve downstream detection performance. Wu et al. [6] proposed SKF, a semantic-aware knowledge-guided framework in which pretrained semantic segmentation guides the restoration of image color and brightness, thereby improving image fidelity and semantic consistency. He et al. [7] proposed SWANet, an image enhancement network that combines multiscale attention with frequency-domain optimization to enhance

✉ Mingkun Feng
mkfeng96@zust.edu.cn

[1] School of Artificial Intelligence and Information Engineering, Zhejiang University of Science and Technology, Hangzhou 310023, Zhejiang, China

[2] School of Computer Science and Technology, Zhejiang University of Science and Technology, Hangzhou 310023, Zhejiang, China

Published online: 15 September 2026

illumination while suppressing low-light noise. Although these methods can improve visual quality, the optimization objectives of image enhancement are not fully aligned with object detection metrics. Moreover, an independent preprocessing module introduces additional computational latency, making it difficult to satisfy the stringent real-time requirements of autonomous driving.

Another line of research focuses on end-to-end detection under adverse illumination. Du et al. [8] introduced DAI-Net, which performs day-night domain adaptation at the feature level to reduce distribution gaps across lighting conditions. Wang et al. [9] proposed Unsupervised Illumination Adaptation (UIA) for low-light vision, providing illumination-consistent representations that can benefit downstream perception. For in-vehicle scenes with strong local light interference, Hui et al. [10] proposed WSA-YOLO, a weakly supervised and adaptive detection framework that integrates illumination decomposition and enhancement into the detector. Wu et al. [11] further developed WTEFNet, which combines low-light enhancement, wavelet-based feature extraction, and adaptive fusion; by separating high- and low-frequency components, it suppresses noise while preserving structural details. Although these methods have improved low-light detection performance, practical deployment still involves clear trade-offs. On one hand, global enhancement alone is often insufficient when local glare is severe and illumination is highly nonuniform. On the other hand, many methods rely on heavy cross-scale fusion to recover weak features, which increases parameters and computation and makes real-time deployment on edge devices, such as the NVIDIA Jetson series, more difficult.

Taken together, these studies have made clear progress in low-light detection, but achieving such gains with a compact detector remains difficult. Enhancement-based pipelines can make dark images visually clearer; however, visual restoration and object detection do not always favor the same features, and the additional preprocessing network increases inference latency. End-to-end detectors reduce this mismatch, but the recovery of weak targets often relies on heavier fusion or attention structures. Therefore, this work focuses on a lightweight end-to-end design in which illumination adaptation, noise suppression, and weak-feature preservation are handled inside the detector without unnecessary parameter growth.

To address the above issues, we propose LUMA-YOLO, a lightweight illumination-adaptive object detection algorithm based on the recently released YOLO26, to balance detection accuracy and inference speed. Systematic optimizations are introduced at the input, feature extraction, and feature fusion stages of the network.

Specifically, a Lightweight Illumination-Adaptive Module (LIAM) is embedded at the network input to guide the model in perceiving global illumination information, thereby mitigating the effects caused by differences between daytime and nighttime environments. Second, Average Pooling Convolution (AConv) is adopted to replace conventional convolution, so that high-frequency noise is suppressed while texture details of small targets are preserved through smoother local aggregation. In addition, the C3k2 module is reconstructed by replacing its C3K component with MMB, thereby improving the extraction and preservation of target contour information. To further enhance global-context perception, the redundancy-prone C2PSA module in the original YOLO26 model is removed, and the conventional SPPF layer is replaced with the SPPELAN structure. This design not only effectively controls the number of model parameters but also improves multiscale feature aggregation and prevents the loss of weak semantic information in deep layers. Finally, the Simple Parameter-Free Attention Module (SimAM) is embedded in the multiscale feature fusion stage, enabling the model to strengthen target feature responses without introducing additional computational overhead.

Unlike methods that enhance images before detection, LUMA-YOLO couples illumination calibration with the detector. LIAM is, therefore, optimized by the detection objective, rather than only by visual enhancement quality, which helps reduce the mismatch between image restoration and object localization. In the backbone, AConv and MMB are designed for common low-light degradation. AConv weakens noisy local responses during downsampling, and MMB uses lightweight memory-guided modulation to preserve weak contours under different illumination conditions. In the neck, the redundancy-prone C2PSA module is removed, while SPPELAN and parameter-free SimAM are used to strengthen multiscale feature responses without adding learnable parameters. These design effects are reflected in the ablation results: on the NOD dataset, the complete model improves mAP@0.5 and mAP@0.5:0.95 from 0.631 and 0.418 to 0.657 and 0.434, respectively, while keeping only 2.47 M parameters. Results on the ExDark, NOD, and TDND datasets further show that LUMA-YOLO improves detection performance over recent YOLO-based, adverse-condition, and lightweight transformer-based detectors while maintaining a compact model size.

In summary, the main contributions of this paper are as follows:

1. We introduce LIAM as an input-level illumination adaptation module for end-to-end low-light detection. Unlike independent enhancement preprocessing, LIAM is trained together with the detector, so illumination calibration is directly guided by the detection task.
2. We improve low-light feature extraction by combining AConv with the proposed MMB. AConv reduces noise

interference during feature downsampling, while MMB uses lightweight memory-guided modulation to retain weak contours and texture information under different illumination conditions.
3. We refine the neck structure to balance accuracy and model complexity. Specifically, C2PSA is removed, SPPF is replaced with SPPELAN, and SimAM is added after multiscale feature concatenation to enhance foreground responses without introducing additional learnable parameters.
4. We evaluate LUMA-YOLO on the ExDark, NOD, and TDND datasets through comparisons with recent lightweight detectors, ablation and repeated-run analyses, qualitative results, Grad-CAM visualization, Jetson deployment, and failure-case analysis.

2 Related Work

2.1 Low-Light Image Enhancement

Research on low-light image enhancement (LLIE) has gradually evolved from an early emphasis on visual-quality improvement to joint optimization with downstream tasks such as object detection. Although early deep learning methods improved overall image brightness, they often overlooked regional content differences, leading to color distortion in processed low-light images. To alleviate this problem, Wu et al. [6] proposed SKF, a semantic-aware knowledge-guided framework in which semantic priors were introduced into the enhancement network and a dedicated color-histogram loss was designed to preserve color consistency across different instance regions. Yan et al. [12] proposed the HVI color space for low-light scenes, in which brightness and chromaticity are decoupled to effectively suppress color noise during enhancement. Zhou et al. [13] proposed GLARE, which improves image fidelity in low-light scenes through generative latent feature alignment and codebook retrieval.

However, such fully supervised models rely heavily on paired datasets containing both low-light and normal-light images. To overcome this limitation, unsupervised and weakly supervised methods have been increasingly explored. PairLIE, proposed by Fu et al. [14], removes the dependence on normally illuminated reference images. By employing classical Retinex theory to constrain reflectance consistency, it achieves high-quality enhancement and denoising at relatively low computational cost. Tang et al. [15] proposed a two-stage end-to-end network, Zero-LEINR, in which illumination enhancement and image denoising are divided into two independent tasks, thereby mitigating interference from inherent noise during low-light brightening.

2.2 DETR-Based Object Detectors

Transformer-based detectors have emerged as an important research direction in generic object detection. DETR [4] was the first to introduce the Transformer encoder-decoder architecture and bipartite matching loss into object detection. By formulating detection as a set prediction problem, it eliminated heuristic post-processing steps such as anchor design and non-maximum suppression. Nevertheless, the original DETR suffers from slow convergence, long training schedules, and high computational cost, which makes it difficult to apply directly to real-time detection scenarios. To address these limitations, Deformable DETR [16] introduced deformable attention, which enables the model to focus on only a small number of critical sampling points, thereby reducing the computational burden of global attention while improving multiscale feature modeling. Conditional DETR [17], Anchor DETR [18], and DAB-DETR [19] further improved DETR from the perspective of query design by introducing more explicit positional priors and dynamic anchor representations, making object queries easier to optimize. Subsequently, DN-DETR [20] and DINO [21] further accelerated convergence and significantly improved the detection accuracy of DETR-like detectors through denoising training and more stable query initialization strategies.

Recent studies have further pushed DETR-like detectors toward real-time and lightweight deployment. Zhao et al. [22] proposed RT-DETR, which combines an efficient hybrid encoder with IoU-aware query selection to reduce redundant computation and improve inference efficiency. Lv et al. [23] proposed RT-DETRv2, where selective multi-scale feature extraction, deployment-friendly sampling, and refined training strategies are introduced to obtain a stronger real-time baseline. Chen et al. [24] proposed LW-DETR, which compresses the detection architecture by combining a lightweight ViT encoder, feature projection, and a shallow DETR decoder. Peng et al. [25] proposed D-FINE, which redefines bounding-box regression in DETRs as fine-grained distribution refinement to improve localization quality while maintaining real-time efficiency. Although these methods have significantly improved the efficiency of Transformer-based detectors, they usually still require more complex training recipes, higher memory consumption, and stronger hardware support than compact one-stage detectors. In addition, most of them are designed and evaluated under general illumination conditions, with limited consideration of weak textures, strong noise, and local glare in low-light driving scenes. These limitations indicate that low-light autonomous driving detection still requires a compact detector with lower training and deployment cost, as well as explicit illumination adaptation.

2.3 Low-Light Object Detection

Single-stage object detection algorithms represented by YOLO have been widely used in engineering practice because of their favorable balance between inference speed and accuracy. In recent years, this family of detectors has evolved rapidly. Ultralytics released YOLO11 [26], reporting an improved speed-accuracy trade-off for practical deployment. YOLOv12 [27] moved toward an attention-centric design to strengthen global-context modeling while preserving real-time performance. YOLOv13 [28] introduced hypergraph-enhanced adaptive correlation modeling for more effective feature interaction under complex scenes. More recently, Ultralytics YOLO26 [29] was released as a unified real-time end-to-end vision model family, including NMS-free object detection for efficient deployment. Although these architectures perform stably under normal illumination, their performance often degrades in low-light road environments, where target contrast is weak and vehicle or pedestrian features can be easily suppressed as noise. To support more systematic evaluation under such conditions, DarkVision [30] provides a multi-illuminance and multi-camera benchmark for low-light image/video perception, while MPE-DETR [31] explores multiscale pyramid enhancement with RT-DETR for low-light object detection. Current research on low-light detection mainly follows two directions.

One direction focuses on optimizing internal network features. For example, FeatEnHancer [32] hierarchically enhances multiscale representations for downstream detection under low-light vision. YOLA [33] learns illumination-invariant features through Lambertian-inspired constraints and improves robustness under severe illumination variation. NLE-YOLO [34] introduces low-frequency filtering and attention-based feature extraction to suppress noise while preserving discriminative structures. In addition, WSA-YOLO [10] integrates illumination decomposition and adaptive enhancement into the detection pipeline for scenes with strong local light interference. Overall, these methods guide the detector to learn more robust feature representations from degraded low-light inputs, thereby improving detection performance in challenging illumination environments.

The other direction focuses on constructing image enhancement and illumination-adaptive modules. Such methods usually employ an enhancement model or an illumination-calibration branch to improve the signal-to-noise ratio of the input image or intermediate features, thereby reducing the difficulty of downstream detection. To reduce the dependence on low-light annotations, the zero-shot day-night domain adaptation framework in [8] learns illumination-invariant representations from Retinex-inspired constraints. WTEFNet [11] combines low-light enhancement, wavelet-based feature extraction, and adaptive fusion to retain structural details in advanced driver-assistance scenarios. ILENet [35] uses an illumination-modulated Laplacian-pyramid enhancement network and couples enhancement with detection through end-to-end optimization. Related adverse-condition detectors also provide useful references: IA-YOLO [36] learns image-adaptive processing parameters for detection under degraded weather, and GDIP [37] introduces differentiable image-processing operations into the detection pipeline. These methods provide useful support for low-light road perception. However, it remains difficult to simultaneously preserve weak target features, suppress noise, and maintain a compact model for real-time autonomous driving deployment.

3 Methodology

3.1 Overall Framework

As shown in Fig. 1, LUMA-YOLO is built on YOLO26 and retains its one-stage detection paradigm for low-light driving scenes. The framework contains an illumination-adaptation front end, a lightweight backbone, a multiscale feature fusion neck, and a detection head. Instead of enhancing the input image with a separate preprocessing network, LIAM is optimized together with the detector, so that illumination calibration is directly coupled with the detection objective.

LIAM is inserted before the first convolutional layer of the backbone, and its output is then sent to the backbone for hierarchical feature extraction. In the backbone, AConv is used in the three main downsampling stages that build the feature hierarchy passed to the neck. Placing AConv at these transitions smooths noise when the spatial resolution is reduced, while leaving the overall YOLO26 hierarchy unchanged. For feature extraction, the C3K component in the backbone feature blocks is replaced by the proposed MMB; the resulting blocks are denoted as MGFM and are used across the backbone stages.

After the last backbone stage, the original SPPF layer is replaced by SPPELAN to enrich multiscale context aggregation. The neck keeps the top-down and bottom-up fusion route of YOLO26. High-level semantic features are first upsampled and merged with lower-level features; the fused representations are then propagated through the bottom-up path. In this bottom-up route, AConv is used before the intermediate- and high-level fusion stages, and SimAM is applied only after the corresponding concatenation operations, before the following C3k2 refinement blocks. The detection head finally predicts objects from the three refined feature levels for small, medium, and large targets.

This arrangement is mainly determined by the degradation process in low-light images. LIAM is placed before the

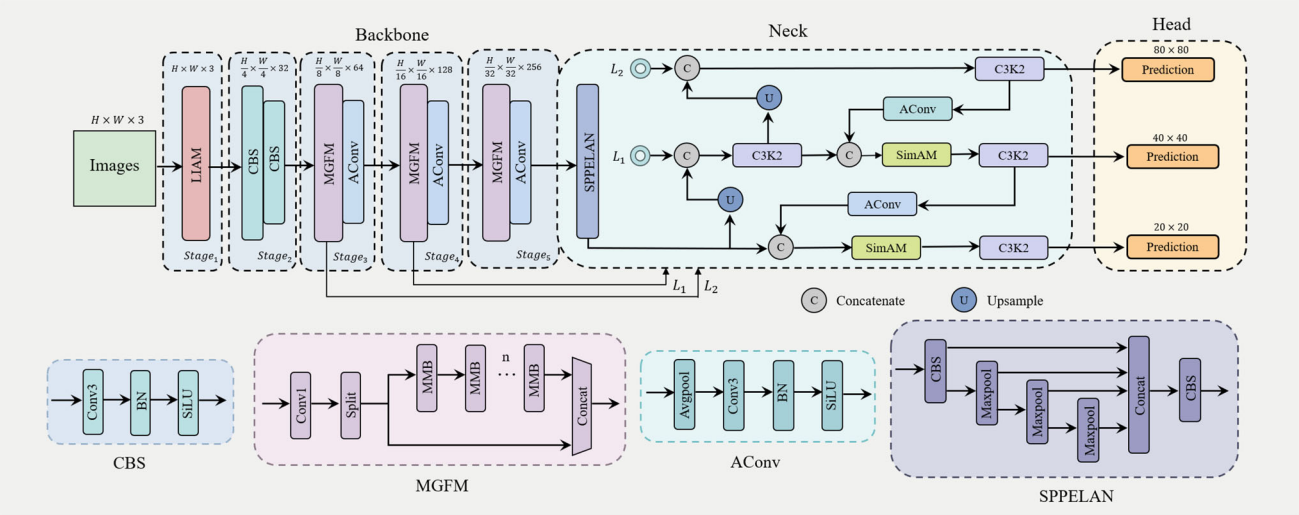


Fig. 1 Overall architecture of the proposed LUMA-YOLO framework

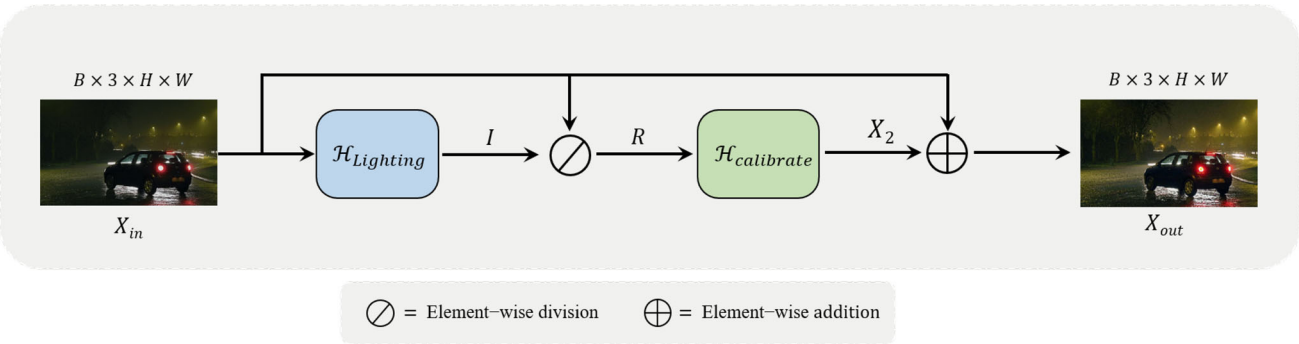


Fig. 2 Overall architecture of the proposed LIAM

first convolutional layer of the backbone so that illumination imbalance and color shift can be handled before the image is repeatedly downsampled. Otherwise, weak edges of small or distant targets may be further suppressed in the early feature maps. AConv is used at the main downsampling transitions because these layers determine which local responses are retained when the spatial resolution decreases; smoothing at this stage helps reduce noise propagation without changing the backbone hierarchy. MMB is kept in the backbone blocks, since weak contours and illumination-dependent features need to be encoded before they are sent to the neck for multiscale fusion. In the neck, SPPELAN is used after the deepest backbone feature to provide richer context, while SimAM is inserted only after the selected bottom-up concatenations, where shallow texture noise and high-level semantic features are directly mixed. This placement avoids applying attention at every fusion step and keeps the feature fusion stage lightweight.

3.2 The Proposed LIAM

SCI [38] proposed a lightweight self-calibrated illumination learning network, which achieved efficient low-light image enhancement with minimal computational overhead through a weight-shared cascaded architecture and a no-reference loss function. Inspired by this work, and to address uneven low-light distribution in autonomous driving scenarios as well as stringent real-time requirements, we design an end-to-end Lightweight Illumination-Adaptive Module, termed LIAM, as shown in Fig. 2.

Inspired by SCI [38], LIAM follows a Retinex-inspired decomposition and is designed to perform low-light enhancement and feature calibration for the input image, as formulated below:

$$
\begin{aligned}
I &= \mathcal{H}_{\text{Lighting}}(X_{\text{in}}) \\
R &= X_{\text{in}} \oslash I \\
X_{\text{out}} &= X_{\text{in}} + \mathcal{H}_{\text{calibrate}}(R)
\end{aligned}
\tag{1}
$$

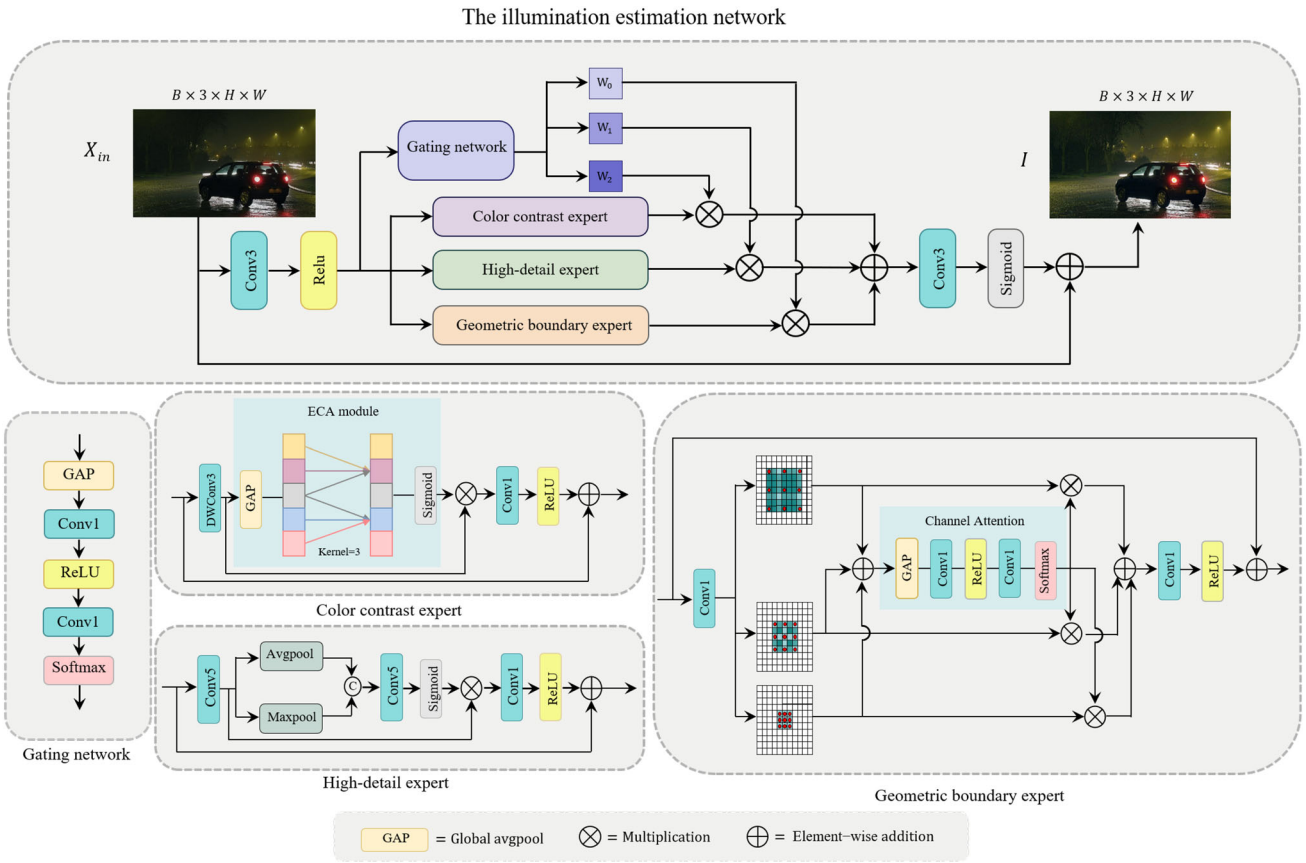


Fig. 3 Overall architecture of the proposed illumination estimation network

where $\mathcal{H}_{\text{Lighting}}$ denotes the illumination estimation network, and $\mathcal{H}_{\text{calibrate}}$ denotes the self-calibration network. Given the input image $X_{\text{in}} \in \mathbb{R}^{B\times3\times H\times W}$, we first estimate the illumination component $I \in \mathbb{R}^{B\times3\times H\times W}$ using $\mathcal{H}_{\text{Lighting}}$. Next, X_{in} is element-wise divided by I to obtain the reflectance component $R \in \mathbb{R}^{B\times3\times H\times W}$. The reflectance component R is then fed into the self-calibration network $\mathcal{H}_{\text{calibrate}}$, and the calibrated residual is added back to the original input X_{in}. This process yields the enhanced output image $X_{\text{out}} \in \mathbb{R}^{B\times3\times H\times W}$.

3.2.1 The Proposed Illumination Estimation Network

Conventional single convolutions often struggle to handle local color shifts and detail loss in low-light images. To this end, an illumination estimation network, denoted by $\mathcal{H}_{\text{Lighting}}$, was designed based on a dense mixture-of-experts architecture, as shown in Fig. 3. In this module, the input image X_{in} is first passed through a 3×3 convolution followed by a ReLU activation function to obtain the shallow feature map $X'_{\text{in}} \in \mathbb{R}^{B\times3\times H\times W}$, which is then fed in parallel into the mixture-of-experts network and the gating network. The mixture-of-experts network consists of three expert branches for feature extraction, while the gating network employs global average pooling, convolution, and activation functions to obtain global feature information from X'_{in} and uses a softmax function to generate dynamic weighting coefficients $W_i \in \mathbb{R}^{B\times1\times1}$, where $i \in \{0, 1, 2\}$.

The three expert branches are designed to extract different features from the shallow feature map, as described below:

Color contrast expert: The color contrast expert is introduced to alleviate color distortion in low-light images. The corresponding formulation is given as follows:

$$\begin{aligned} F_d &= \text{DWConv3}(X'_{\text{in}}) \\ A_c &= \sigma(\text{Conv1D}_{k=3}(\text{GAP}(F_d))) \\ F_{\text{cc}} &= \text{ReLU}(\text{Conv1}(F_d \otimes A_c)) + X'_{\text{in}} \end{aligned} \tag{2}$$

where σ denotes the Sigmoid function, and GAP denotes global average pooling. Given the input feature map X'_{in}, local spatial responses are first extracted by a 3×3 depthwise convolution. Channel dependencies are then modeled by an Efficient Channel Attention (ECA) module [39], in which global average pooling and a one-dimensional convolution with a kernel size of 3 are used to generate channel-wise attention weights. The resulting attention map is applied to

recalibrate the intermediate feature representation. Finally, a 1×1 convolution followed by a ReLU activation is performed, and a residual connection is introduced to obtain the color contrast feature map $F_{\text{cc}} \in \mathbb{R}^{B\times 3\times H\times W}$.

High-detail expert: The high-detail expert is used to preserve weak texture details in local low-light regions. The corresponding formulation is given as follows:

$$\begin{aligned} F_l &= \text{Conv5}(X'_{\text{in}}) \\ M_s &= \sigma(\text{Conv5}([\text{AvgPool}(F_l), \text{MaxPool}(F_l)])) \otimes F_l \\ F_{\text{hd}} &= \text{ReLU}(\text{Conv1}(M_s)) + X'_{\text{in}} \end{aligned} \tag{3}$$

The input feature map X'_{in} is first processed by a 5×5 depthwise convolution to enlarge the receptive field. Channel-wise average pooling and max pooling are then performed on F_l to generate two spatial descriptors, which are concatenated and passed through a 5×5 convolution followed by a Sigmoid activation to produce a spatial attention map. This attention map is used to enhance fine-grained detail responses. After that, a 1×1 convolution and a ReLU activation are applied, and the refined feature is added back to the input through a residual connection to obtain the high-detail feature map $F_{hd} \in \mathbb{R}^{B\times 3\times H\times W}$.

Geometric boundary expert: The geometric boundary expert is introduced to capture multiscale structural boundary information in low-light images. The corresponding formulation is given as follows:

$$\begin{aligned} X_r &= \text{Conv1}(X'_{\text{in}}) \\ V_i &= \text{DWConv3}_{d=i}(X_r), \quad i \in \{1, 2, 3\} \\ U &= V_1 + V_2 + V_3 \\ [A_1, A_2, A_3] &= \text{Softmax}(\phi(U)) \\ V_{\text{fuse}} &= A_1 \otimes V_1 + A_2 \otimes V_2 + A_3 \otimes V_3 \\ F_{\text{gb}} &= \text{ReLU}(\text{Conv1}(V_{\text{fuse}})) + X'_{\text{in}} \end{aligned} \tag{4}$$

The input feature map X'_{in} is first compressed by a 1×1 convolution to obtain a compact representation. Three parallel depthwise dilated convolutions with dilation rates of 1, 2, and 3 are then employed to extract boundary responses at different scales. The resulting multiscale features are aggregated into U, and branch-aware attention weights are generated by applying global average pooling to U, followed by two 1×1 convolutions, a ReLU activation, and a Softmax operation. The three branches are subsequently fused in an adaptive manner, and the fused feature is projected back to the original channel dimension by a 1×1 convolution. Finally, a ReLU activation and a residual connection are applied to produce the geometric boundary feature map $F_{\text{gb}} \in \mathbb{R}^{B\times 3\times H\times W}$.

The output feature maps generated by the three experts are then combined through a weighted summation using the weights produced by the gating network. Finally, a 3×3 convolution and a Sigmoid activation function are applied, and a residual connection is established with the original input to obtain the illumination component map $I \in \mathbb{R}^{B\times 3\times H\times W}$.

The corresponding formulation is given as follows:

$$\begin{aligned} F_{\text{fuse}} &= W_0 \otimes F_{\text{cc}} + W_1 \otimes F_{\text{hd}} + W_2 \otimes F_{\text{gb}} \\ I &= \sigma(\text{Conv3}(F_{\text{fuse}})) + X_{\text{in}} \end{aligned} \tag{5}$$

3.2.2 The Proposed Self-calibration Network

Although the reflectance component R is obtained through element-wise division, noise is still inevitably introduced in some low-light regions of the image, resulting in artifacts and color shifts in R. To address this issue, a cascaded self-calibration network, $\mathcal{H}_{\text{calibrate}}$, was employed, as shown in Fig. 4.

For the noisy reflectance feature $R \in \mathbb{R}^{B\times 3\times H\times W}$, the module first performs a preliminary extraction of shallow spatial textures from the input features by using a 3×3 depthwise separable convolution, BN normalization, and a Sigmoid activation function. The network then passes through a cascade of k calibration units with identical structures. Each calibration unit contains two consecutive blocks, in which a 3×3 depthwise convolution and a 1×1 pointwise convolution are sequentially performed, followed by BN normalization and a ReLU activation function. In addition, residual connections based on element-wise addition are introduced between adjacent calibration units. The generation process of the output feature F_t of the t-th calibration unit is described as follows:

$$F_{t+1} = F_t + \text{Unit}(F_t), \ t = 0, 1, \ldots, k-1 \tag{6}$$

After passing through the k cascaded units, the deep features are fed into a 3×3 depthwise separable convolution followed by a Sigmoid activation function. The resulting calibration map is then subtracted element-wise from the original reflectance feature R. In this way, the calibration component is explicitly separated from R, yielding a high-signal-to-noise-ratio output feature map $X_{\text{out}} \in \mathbb{R}^{B\times 3\times H\times W}$.

3.2.3 Joint Loss Optimization Function

In the conventional YOLO architecture, the loss function focuses on bounding-box regression and category prediction only, and, therefore, cannot directly guide the illumination-enhancement module. Moreover, two-stage pipelines often introduce inconsistent optimization objectives between image enhancement and object detection. To achieve end-to-end joint optimization, we adopt the no-reference illumination constraint from SCI and incorporate its loss term L_{sci} into

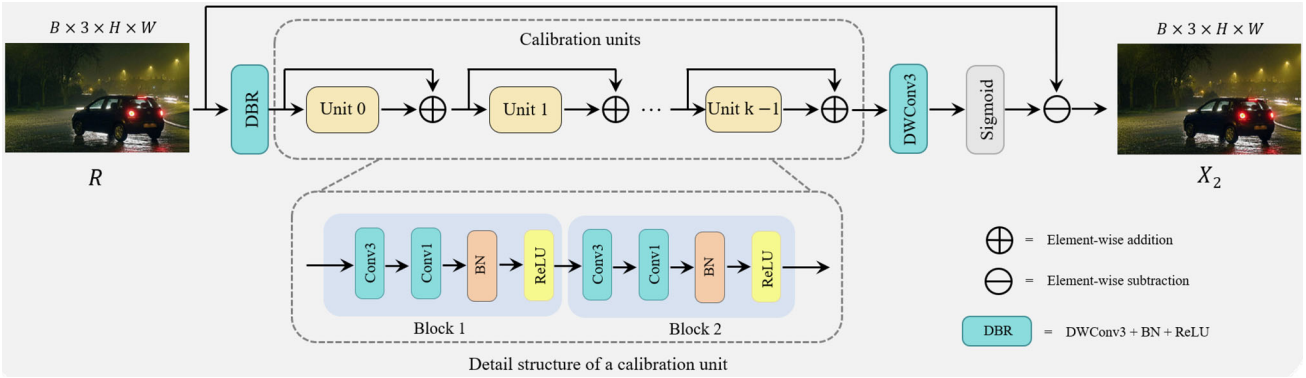


Fig. 4 Overall architecture of the proposed self-calibration network

the overall objective of LUMA-YOLO, as follows:

$$L_{\text{loss}} = L_{\text{det}} + \lambda L_{\text{sci}} \tag{7}$$

where L_{det} denotes the default detection loss in YOLO26, and λ is a balancing coefficient that controls the contribution of L_{sci}. This design differs from the original SCI setting, where the loss is applied only to image enhancement. In contrast, the joint objective enables cross-task information interaction, effectively mitigating optimization inconsistency in two-stage frameworks and improving robustness and localization accuracy in low-light autonomous driving scenarios.

3.3 The Proposed MMB Module

To address uneven local brightness and insufficient feature extraction in low-light scenes, we replace the C3K component inside the backbone C3k2-style feature blocks with the proposed Memory-Modulated Bottleneck (MMB). In the final model, these modified backbone blocks are denoted as MGFM and are used across the main backbone stages from low-level to high-level feature extraction. The C3k2 blocks in the neck are kept unchanged so that the effect of MMB is concentrated on backbone feature representation rather than feature fusion. The MMB consists of three branches: a dynamic computation branch, a memory branch, and an adaptive fusion branch. Its overall architecture is shown in Fig. 5.

The dynamic computation branch adopts a standard bottleneck structure to extract local spatial features, as shown below:

$$X_{\text{dy}} = \text{BN}(\text{Conv3}(\text{SiLU}(\text{BN}(\text{Conv1}(X))))) \tag{8}$$

where Conv1 and Conv3 denote 1×1 and 3×3 convolutions, respectively, while BN and SiLU denote batch normalization and the SiLU activation function. The input feature $X \in \mathbb{R}^{B\times C\times H\times W}$ is first compressed by $Conv1$-BN-SiLU and then processed by Conv3 and BN to extract local spatial information, yielding the dynamic spatial feature $X_{\text{dy}} \in \mathbb{R}^{B\times C\times H\times W}$.

The memory branch is used to extract global feature information under different illumination conditions. It maintains a set of learnable prototypes $P_j \in \mathbb{R}^{C\times 1\times 1}$, where $j \in \{0, 1, \ldots, n-1\}$. Each prototype is updated during backpropagation. To adaptively assign prototype weights according to the illumination condition of the input image, a lightweight gating network is introduced:

$$G = \text{Softmax}(\text{Conv1}(\text{GAP}(X))) \tag{9}$$

where GAP denotes global average pooling. The input feature X is first pooled to aggregate spatial context and then projected by a 1×1 convolution to $\mathbb{R}^{B\times n\times 1\times 1}$. After Softmax normalization, branch weights $G_j \in \mathbb{R}^{B\times 1\times 1\times 1}$ are obtained. These weights are multiplied with the prototypes P_j to compute the global feature vector $M \in \mathbb{R}^{B\times 1\times 1}$:

$$M = \sum_{j=0}^{n-1} G_j \otimes P_j \tag{10}$$

To effectively fuse the global feature M with the dynamic spatial feature X_{dy}, an adaptive fusion branch is used:

$$\beta = \sigma(\alpha)$$
$$X_{\text{fuse}} = \beta \otimes X_{\text{dy}} + (1-\beta) \otimes M \tag{11}$$

where α denotes a learnable parameter, and $\beta \in (0, 1)$ is obtained through the Sigmoid function. During computation, M is broadcast to the same spatial size as X_{dy}. The gating weight β controls the retention ratio between local dynamic features and global memory features. Under relatively favorable illumination, the network tends to assign a larger β to preserve local details and edges. Under extremely low-light conditions, the network adaptively decreases β to introduce more global memory information, thereby enhancing target responses in low-light regions.

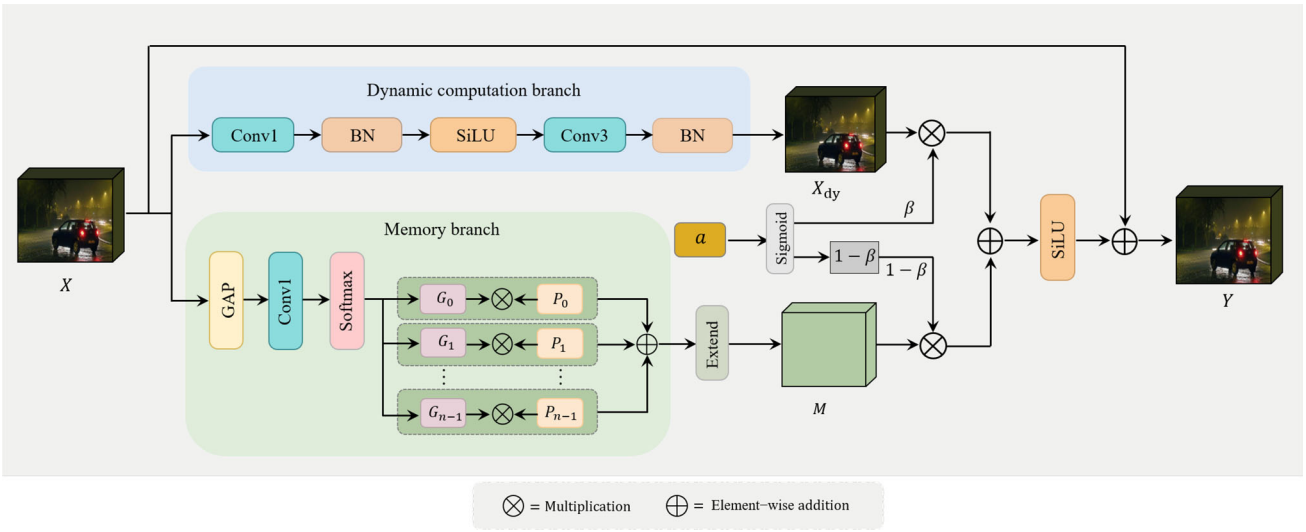


Fig. 5 Overall architecture of the proposed MMB module

Finally, the fused feature $X_{\text{fuse}} \in \mathbb{R}^{B\times C\times H\times W}$ is activated by SiLU and added to the input feature X through a residual connection to obtain the output $Y \in \mathbb{R}^{B\times C\times H\times W}$:

$$Y = X + \text{SiLU}(X_{\text{fuse}}) \tag{12}$$

In summary, while maintaining a lightweight structure, the MMB module effectively enhances the network's ability to extract and represent low-light features, providing an efficient feature-processing solution for object detection.

3.4 SimAM

During the construction of the feature pyramid network, the neck of YOLO26 uses feature concatenation for multiscale information fusion. Although effective under normal illumination, in low-light scenes the deep target semantics are often weak, while background noise remains in shallow features. As a result, direct concatenation tends to mix shallow noise with weak deep features, reducing the signal-to-noise ratio of the fused feature maps and leading to missed detections and false detections.

To alleviate the feature interference caused by multiscale fusion in low-light scenes and enhance weak target perception, SimAM is introduced after selected feature concatenation operations in the neck. Specifically, it is placed after the two bottom-up concatenations that fuse intermediate and high-level features, before the subsequent C3k2 refinement blocks. SimAM is not inserted after the top-down concatenations, which avoids applying attention repeatedly at every fusion step and keeps the neck lightweight. Unlike conventional channel attention mechanisms such as the SE module [40] or spatial attention mechanisms such as CBAM [41], the Simple Parameter-Free Attention Module (SimAM)

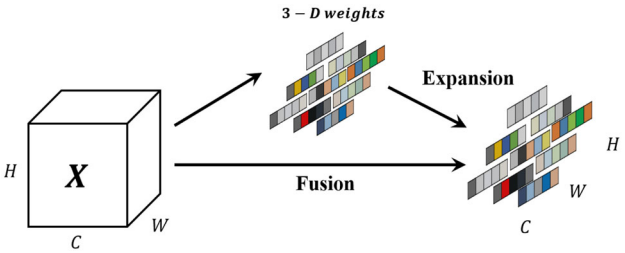


Fig. 6 Overall architecture of the SimAM

[42] is based on the theory of spatial inhibition in neuroscience. Its overall architecture is shown in Fig. 6. By defining an energy function, it infers full three-dimensional attention weights from the current feature map without introducing additional learnable parameters.

For a target neuron t in the feature map and the other neurons x_i within the same channel, the energy function is defined as follows:

$$e_t = \frac{4(\hat{\sigma}^2 + \lambda)}{(t - \hat{\mu})^2 + 2\hat{\sigma}^2 + 2\lambda}$$

$$\hat{\mu} = \frac{1}{M}\sum_{i=1}^{M} x_i$$

$$\hat{\sigma}^2 = \frac{1}{M}\sum_{i=1}^{M}(x_i - \hat{\mu})^2 \tag{13}$$

where $\hat{\mu}$ and $\hat{\sigma}^2$ denote the sample mean and variance of all neurons in the channel, respectively; M denotes the number of pixels in the current feature map; and λ denotes the regularization coefficient controlling inhibition strength. The smaller the value of e_t, the more distinct neuron t is from surrounding background neurons, and the greater its importance.

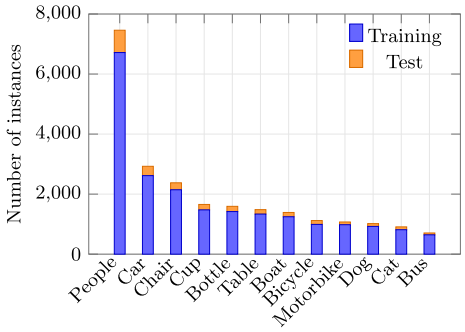


Fig. 7 Class-wise distribution of ExDark image samples in the training and test splits

Therefore, the importance weight can be obtained from $1/e_t$, and the final SimAM enhancement process is expressed as follows:

$$\tilde{X} = \text{Sigmoid}\left(\frac{1}{e_t}\right) \odot X \tag{14}$$

where $\odot$ denotes element-wise multiplication. This parameter-free design enhances responses to weak targets under low-light conditions and suppresses background noise without increasing model parameters or FLOPs.

4 Experiments

4.1 Datasets

To comprehensively evaluate the performance of LUMA-YOLO in low-light road object detection tasks, experiments were conducted on three public datasets: ExDark, NOD, and TDND.

ExDark dataset

The ExDark dataset was introduced by Loh and Chan [43] for low-light object detection. The original release contains 7363 images across 12 categories. In this work, we follow a cleaned split with 7328 images, including 6595 training images and 733 test images, with resolutions ranging from 640 × 480 to 1920 × 1080 pixels. The class-wise annotated-object instance distribution of this split is shown in Fig. 7.

NOD dataset

The NOD (Night Object Detection) dataset was proposed by Morawski et al. [44] for extreme low-light road-scene detection. The full benchmark provides more than 7000 annotated images in three categories. In our experiments, we use a 4006-image subset, split into 3,206 training images and 800 test images, at a resolution of 3936 × 2624 pixels. The class-wise image-sample distribution of this subset is shown in Fig. 8.

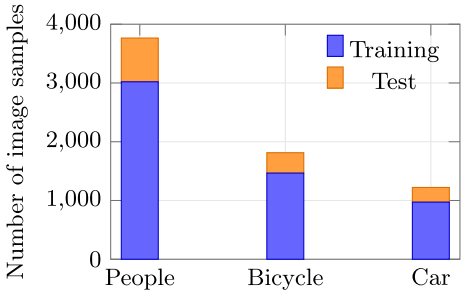


Fig. 8 Class-wise distribution of NOD image samples in the training and test splits

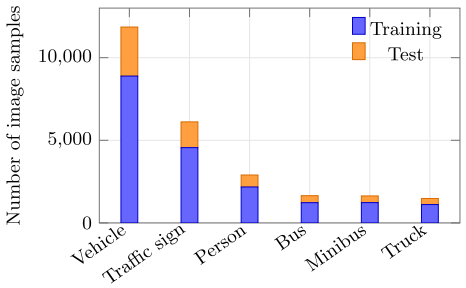


Fig. 9 Class-wise distribution of TDND image samples in the training and test splits

TDND dataset

The TDND (Transnational image object detection datasets from nighttime driving) benchmark was introduced by Nie et al. [45]. The original release is a large-scale multinational nighttime-driving dataset with challenging illumination and weather conditions. In this study, we use a curated subset of 12,773 nighttime-driving images covering six categories, with 9579 images for training and 3194 images for testing, at a resolution of 1280 × 720 pixels. The class-wise image-sample distribution of this subset is shown in Fig. 9.

4.2 Experimental Settings

Model training and the main benchmark evaluation were conducted using the PyTorch 2.9.1 deep learning framework. The desktop experimental platform was equipped with an Intel Core E5-2666 v3 CPU and an RTX 5060 Ti 16 GB GPU. The input image size was uniformly set to 640 × 640, and the batch size was set to 32. Stochastic gradient descent (SGD) was adopted as the optimizer, with an initial learning rate of 0.01, a momentum of 0.937, and a weight decay of 0.0005. Cosine annealing was adopted as the learning rate scheduling strategy, by which the learning rate was gradually reduced during training to improve model convergence. The model was trained for a total of 290 epochs, of which the first 3 epochs were used for learning rate warm-up. This strategy enabled a smooth transition during the early stage of training

and prevented training instability caused by an excessively high initial learning rate.

All datasets were converted to the YOLO annotation format before training. During training and evaluation, images from different datasets were processed with the same resizing and normalization procedure, and the input size was fixed at 640 × 640. For a fair comparison, the competing detectors were trained and tested on the same splits as LUMA-YOLO, and the same batch size, optimizer, and learning-rate schedule were used whenever applicable. The number of parameters was counted from the model summary under the same input setting. For desktop speed testing, each model was warmed up first and then evaluated with a batch size of 1. FPS was computed from the average forward inference time on the test images, excluding data loading and result visualization. The ablation experiments used the same training and evaluation protocol, with only the corresponding modules changed.

4.2.1 Evaluation Metrics

To comprehensively evaluate model performance, we use precision, recall, F1-score, mean average precision at an IoU threshold of 0.5 (mAP@0.5), mean average precision averaged over IoU thresholds from 0.5 to 0.95 (mAP@0.5:0.95), number of parameters (Params), and frames per second (FPS). The corresponding formulas are:

$$\begin{aligned}
\text{Precision} &= \frac{\text{TP}}{\text{TP}+\text{FP}} \\
\text{Recall} &= \frac{\text{TP}}{\text{TP}+\text{FN}} \\
F1 &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision}+\text{Recall}} \\
\text{mAP@0.5} &= \frac{1}{N}\sum_{i=1}^{N} AP_i^{0.5} \\
\text{mAP@50:95} &= \frac{1}{M}\sum_{k=1}^{M}\left(\frac{1}{N}\sum_{i=1}^{N} P_{50:95}^{i,k}\right)
\end{aligned} \tag{15}$$

where TP represents the number of correctly identified positive instances. FP refers to the number of instances incorrectly identified as positive. FN represents the number of instances incorrectly identified as negative. N denotes the total number of categories. M denotes the number of IoU thresholds, which is 10, spanning from 0.50 to 0.95 in increments of 0.05.

4.2.2 Comparison Methods

To evaluate the effectiveness of LUMA-YOLO, we compare it with several mainstream object detection methods, including YOLOv8 [46], YOLOv10 [47], YOLOv11 [26], YOLOv12 [27], YOLOv13 [28], YOLO26 [29], YOLO-Master [48], and D-FINE [25]. In addition, IA-YOLO [36] and GDIP-YOLO [37], which were proposed for adverse weather or adverse-condition scenarios, are also included for comparison. All accuracy experiments were conducted on the same datasets and under the same training settings to ensure a fair comparison.

As shown in Table 1, LUMA-YOLO delivers the most competitive overall performance on the ExDark dataset, demonstrating the effectiveness of the proposed method for low-light object detection. In terms of precision, D-FINE-N obtains the highest value of 0.774, followed by YOLO26n with 0.768, while LUMA-YOLO remains very close with 0.766. More importantly, LUMA-YOLO attains the highest recall of 0.613, surpassing the second-best method, YOLO-Master, by 0.3 percentage points. This suggests that the proposed network can detect more valid targets in complex dark environments. Benefiting from this stronger recall while maintaining high precision, LUMA-YOLO achieves the best F1-score of 0.681, exceeding the second-best result of 0.661 obtained by D-FINE-N by 2.0 percentage points.

For the representative detection metrics, LUMA-YOLO also ranks first on both mAP@0.5 and mAP@0.5:0.95, reaching 0.709 and 0.451, respectively. Compared with the second-best D-FINE-N, the improvements are 2.4 percentage points and 1.3 percentage points, respectively. These gains indicate that the proposed method improves both target detection and localization accuracy, especially under stricter IoU thresholds. In addition, LUMA-YOLO contains only 2.47 M parameters, the lowest among all compared methods, even smaller than YOLOv13n with 2.50 M parameters. Although its inference speed of 57.8 FPS is lower than the fastest measurements on the RTX 5060 Ti, such as those of YOLO26n and IA-YOLO, it still satisfies real-time deployment requirements. Overall, considering recall, F1-score, mAP, parameter efficiency, and inference speed, LUMA-YOLO achieves a more desirable trade-off between accuracy and efficiency on the ExDark dataset.

The NOD results in Table 2 show a slightly different ranking pattern from ExDark. D-FINE-N gives the highest precision of 0.839, but its recall drops to 0.532. LUMA-YOLO does not pursue the highest precision; instead, it obtains the best recall of 0.556 while keeping precision at 0.800. This is reflected in the F1-score, where LUMA-YOLO reaches 0.656, ahead of YOLO-Master and D-FINE-N by 0.2 and 0.5 percentage points, respectively. For NOD, where many targets are weakly illuminated, this result suggests that the proposed model is less prone to missing valid objects.

The mAP results lead to the same conclusion. LUMA-YOLO achieves 0.657 mAP@0.5, slightly higher than YOLO-Master with 0.654, and obtains 0.434 mAP@0.5:0.95, which is also higher than D-FINE-N with 0.432. The margins

Table 1 Performance comparison on the ExDark dataset. The best and second-best results are indicated in **bold** and *italic*

Method	Input size	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	Params (M)	FPS	Device
YOLOv8n [46]	640×640	0.726	0.551	0.627	0.644	0.387	3.15	54.6	RTX 5060 Ti
YOLOv10n [47]	640×640	0.726	0.536	0.617	0.616	0.379	2.70	49.5	RTX 5060 Ti
YOLOv11n [26]	640×640	0.715	0.580	0.640	0.658	0.411	2.60	44.6	RTX 5060 Ti
YOLOv12n [27]	640×640	0.734	0.596	0.658	0.676	0.425	2.60	40.3	RTX 5060 Ti
YOLOv13n [28]	640×640	0.693	0.568	0.624	0.640	0.395	*2.50*	42.2	RTX 5060 Ti
YOLO26n [29]	640×640	*0.768*	0.563	0.650	0.666	0.414	2.57	**80.1**	RTX 5060 Ti
YOLO-Master [48]	640×640	0.715	*0.610*	0.658	0.684	0.426	2.69	41.5	RTX 5060 Ti
IA-YOLO [36]	640×640	0.739	0.530	0.617	0.613	0.345	7.12	*67.4*	RTX 5060 Ti
GDIP-YOLO [37]	640×640	0.667	0.575	0.618	0.616	0.354	13.49	18	RTX 5060 Ti
D-FINE-N [25]	640×640	**0.774**	0.577	*0.661*	*0.685*	*0.438*	3.73	35.2	RTX 5060 Ti
LUMA-YOLO (Ours)	640×640	0.766	**0.613**	**0.681**	**0.709**	**0.451**	**2.47**	57.8	RTX 5060 Ti

Table 2 Performance comparison on the NOD dataset. The best and second-best results are indicated in **bold** and *italic*

Method	Input size	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	Params (M)	FPS	Device
YOLOv8n [46]	640×640	*0.811*	0.540	0.648	0.637	0.407	3.15	54.5	RTX 5060 Ti
YOLOv10n [47]	640×640	0.803	0.526	0.636	0.627	0.414	2.70	49.6	RTX 5060 Ti
YOLOv11n [26]	640×640	0.778	0.542	0.639	0.628	0.400	2.60	44.5	RTX 5060 Ti
YOLOv12n [27]	640×640	0.791	0.549	0.648	0.643	0.420	2.60	40.3	RTX 5060 Ti
YOLOv13n [28]	640×640	0.807	0.540	0.647	0.637	0.419	*2.50*	42.1	RTX 5060 Ti
YOLO26n [29]	640×640	0.808	0.527	0.638	0.631	0.418	2.57	**80.1**	RTX 5060 Ti
YOLO-Master [48]	640×640	0.799	*0.553*	*0.654*	*0.654*	0.429	2.69	41.4	RTX 5060 Ti
IA-YOLO [36]	640×640	0.801	0.508	0.622	0.587	0.373	7.12	*67.4*	RTX 5060 Ti
GDIP-YOLO [37]	640×640	0.807	0.505	0.621	0.593	0.376	13.49	18	RTX 5060 Ti
D-FINE-N [25]	640×640	**0.839**	0.532	0.651	0.648	*0.432*	3.73	35.2	RTX 5060 Ti
LUMA-YOLO (Ours)	640×640	0.800	**0.556**	**0.656**	**0.657**	**0.434**	**2.47**	57.8	RTX 5060 Ti

are not large, but they are consistent across the two localization metrics. Meanwhile, LUMA-YOLO uses only 2.47 M parameters and runs at 57.8 FPS on the RTX 5060 Ti. Thus, compared with the methods that are either faster or more precise in a single metric, LUMA-YOLO offers a more balanced result on the NOD dataset.

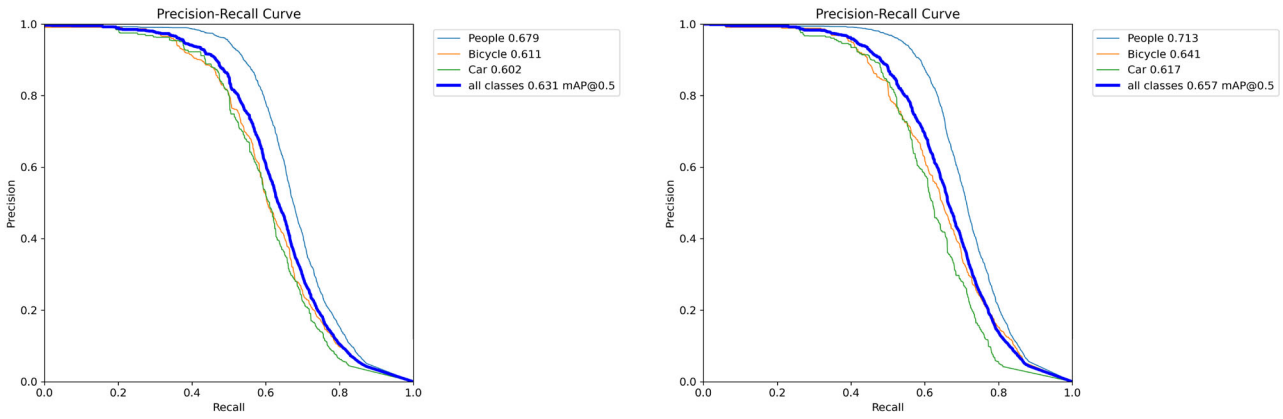
On the TDND dataset, as shown in Table 3, D-FINE-N reports the highest precision of 0.773, whereas LUMA-YOLO gives the highest recall of 0.663. This difference is similar to the trend observed on the NOD dataset: D-FINE-N is more selective in its predictions, while LUMA-YOLO retrieves more valid targets in low-light scenes. The resulting F1-score of LUMA-YOLO is 0.709, which is 1.2 percentage points higher than both YOLO-Master and D-FINE-N. This indicates that the proposed model keeps a better balance between false positives and missed detections on this dataset.

The same tendency can be seen from the mAP metrics. LUMA-YOLO achieves 0.725 mAP@0.5 and 0.461 mAP@0.5:0.95, exceeding D-FINE-N by 0.6 and 0.4 percentage points, respectively. Although the gap is moderate, the improvement appears under both the loose and strict IoU settings, suggesting better localization stability for weak low-light targets. LUMA-YOLO also keeps the smallest parameter count of 2.47 M and runs at 57.8 FPS on the RTX 5060 Ti. Therefore, on the TDND dataset, its advantage is not limited to accuracy alone; it also maintains a compact model size and practical inference speed.

The two PR curves provide a direct visual comparison between the baseline detector and the complete LUMA-YOLO model. In Fig. 10a, the baseline curve drops earlier as recall increases, especially for categories with more challenging low-light appearances, indicating that the original detector is more likely to lose precision when retrieving more positive samples. In Fig. 10b, LUMA-YOLO keeps the class-wise curves closer to the upper-right region over a wider recall range. The overall curve is also smoother and remains higher before the final decline, showing a better balance between missed detections and false positives. This visual comparison suggests that the proposed model produces more

Table 3 Performance comparison on the TDND dataset. The best and second-best results are indicated in **bold** and *italic*

Method	Input size	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	Params (M)	FPS	Device
YOLOv8n [46]	640×640	0.746	0.625	0.680	0.691	0.427	3.15	54.6	RTX 5060 Ti
YOLOv10n [47]	640×640	0.742	0.632	0.682	0.694	0.437	2.70	49.5	RTX 5060 Ti
YOLOv11n [26]	640×640	0.745	0.613	0.673	0.682	0.422	2.60	44.5	RTX 5060 Ti
YOLOv12n [27]	640×640	0.752	0.643	0.693	0.701	0.438	2.60	40.4	RTX 5060 Ti
YOLOv13n [28]	640×640	0.744	0.632	0.683	0.696	0.440	*2.50*	42.1	RTX 5060 Ti
YOLO26n [29]	640×640	*0.766*	0.634	0.694	0.706	0.451	2.57	**80**	RTX 5060 Ti
YOLO-Master [48]	640×640	0.758	*0.644*	*0.697*	0.716	0.451	2.69	41.4	RTX 5060 Ti
IA-YOLO [36]	640×640	0.711	0.553	0.622	0.602	0.368	7.12	*67.4*	RTX 5060 Ti
GDIP-YOLO [37]	640×640	0.716	0.551	0.623	0.599	0.365	13.49	18	RTX 5060 Ti
D-FINE-N [25]	640×640	**0.773**	0.635	*0.697*	*0.719*	*0.457*	3.73	35.2	RTX 5060 Ti
LUMA-YOLO (Ours)	640×640	0.762	**0.663**	**0.709**	**0.725**	**0.461**	**2.47**	57.8	RTX 5060 Ti



(a) Baseline (b) LUMA-YOLO

Fig. 10 PR-curve comparison between the baseline detector and LUMA-YOLO on the NOD dataset

stable confidence responses and preserves more discriminative target features under low-light conditions.

To thoroughly investigate the contribution of each proposed component, ablation experiments were conducted on the NOD dataset under the same training settings with the random seed fixed to 0, and the corresponding results are presented in Table 4. The baseline model is the original detector without LIAM, MMB, or SimAM. It achieves 0.631 mAP@0.5 and 0.418 mAP@0.5:0.95 with 2.57 M parameters. On this basis, different module configurations are introduced step by step to evaluate their individual effectiveness and combined impact on overall detection performance.

First, the effect of each module is examined independently. After introducing LIAM alone, mAP@0.5 increases from 0.631 to 0.639, while mAP@0.5:0.95 rises from 0.418 to 0.425, with the parameter count reduced to 2.43 M. This result indicates that LIAM effectively improves feature representation under low-light conditions while maintaining a compact model size. When MMB is added alone, mAP@0.5 also reaches 0.639 and mAP@0.5:0.95 improves to 0.423, with 2.46 M parameters. Although the gain is slightly smaller than that brought by LIAM under the stricter metric, the improvement still confirms the positive role of MMB in enhancing feature extraction quality. By comparison, using SimAM alone leads to 0.635 mAP@0.5 and 0.417 mAP@0.5:0.95 with 2.42 M parameters. Its contribution is relatively limited when applied independently, suggesting that the attention mechanism is more effective when combined with stronger feature-enhancement modules.

Next, dual-module combinations further reveal complementarity among the proposed components. The joint use of LIAM and MMB improves mAP@0.5 to 0.649 and mAP@0.5:0.95 to 0.432 with 2.47 M parameters, which is clearly better than either module used alone. This result demonstrates that illumination-aware enhancement and feature refinement can work cooperatively to improve detection quality. When MMB and SimAM are combined, the model reaches 0.645 mAP@0.5 and 0.425 mAP@0.5:0.95

Table 5 Repeated-run results for the LIAM-only ablation on the NOD dataset

Seed	P	R	F1	mAP@0.5	mAP@0.5:0.95
0	0.810	0.546	0.652	0.639	0.425
1	0.816	0.541	0.651	0.638	0.428
2	0.828	0.535	0.650	0.636	0.427
3	0.807	0.550	0.654	0.640	0.428

Table 6 Repeated-run results for the MMB-only ablation on the NOD dataset

Seed	P	R	F1	mAP@0.5	mAP@0.5:0.95
0	0.808	0.544	0.650	0.639	0.423
1	0.822	0.543	0.654	0.642	0.425
2	0.821	0.541	0.652	0.641	0.426
3	0.823	0.538	0.651	0.640	0.424

Table 7 Repeated-run results for the SimAM-only ablation on the NOD dataset

Seed	P	R	F1	mAP@0.5	mAP@0.5:0.95
0	0.827	0.527	0.644	0.635	0.417
1	0.820	0.538	0.650	0.636	0.420
2	0.826	0.522	0.640	0.631	0.416
3	0.811	0.539	0.648	0.636	0.418

with 2.46 M parameters, showing that SimAM can further strengthen the effect of MMB. Similarly, the combination of LIAM and SimAM achieves 0.645 mAP@0.5 and 0.428 mAP@0.5:0.95 with only 2.43 M parameters, indicating a favorable balance between performance gain and parameter efficiency.

Finally, after integrating LIAM, MMB, and SimAM simultaneously, the complete LUMA-YOLO model achieves the best overall performance, with mAP@0.5 reaching 0.657 and mAP@0.5:0.95 reaching 0.434. Compared with the baseline, the improvements are 2.6 and 1.6 percentage points, respectively, while the parameter count remains only 2.47 M. These results verify that the three modules are complementary rather than redundant. Overall, the ablation study confirms that each proposed component contributes positively to performance improvement, and their full combination provides the most effective configuration for low-light object detection.

To examine the stability of the single-module ablation results with respect to random initialization, the LIAM-only, MMB-only, and SimAM-only variants were trained four times using different random seeds, while all other experimental settings were kept unchanged. Tables 5, 6, 7 list the precision, recall, F1-score, mAP@0.5, and mAP@0.5:0.95 obtained in each run. For compact presentation, P and R denote precision and recall, respectively. These repeated results provide additional evidence for assessing whether the improvements reported in Table 4 are consistent across different training runs.

Table 5 shows that LIAM brings a consistent improvement across the four repeated runs. The mAP@0.5 values are concentrated between 0.636 and 0.640, and the mAP@0.5:0.95 values range from 0.425 to 0.428. Compared with the baseline results in Table 4, even the lowest LIAM-only run still improves mAP@0.5 by 0.005 and mAP@0.5:0.95 by 0.007. This indicates that the gain of LIAM is not an isolated result from one favorable initialization, but a stable contribution to low-light feature enhancement.

The results in Table 6 further show that the effect of MMB is repeatable. Across the four seeds, mAP@0.5 changes only from 0.639 to 0.642, while mAP@0.5:0.95 changes from 0.423 to 0.426. These results are consistently higher than the baseline values of 0.631 and 0.418, with minimum gains of 0.008 and 0.005, respectively. Therefore, the improvement from MMB can be attributed to its memory-modulated feature extraction design rather than to random variation during training.

Table 7 presents a different case. The mAP@0.5 values remain close to the baseline, ranging from 0.631 to 0.636, but the mAP@0.5:0.95 values vary from 0.416 to 0.420. Since the baseline mAP@0.5:0.95 is 0.418, SimAM alone does not consistently improve the stricter localization metric. This suggests that its independent contribution is limited; its main benefit is more likely to appear when it is used as a refinement module together with LIAM and MMB, as reflected by the full model in Table 4.

Figure 11 illustrates the object detection results of the compared methods on the NOD dataset. To comprehensively evaluate the detection performance of LUMA-YOLO, four representative low-light scenarios are selected: an extremely dark environment with a single bicycle, a dimly lit staircase, a dark park scene with multiple pedestrians, and a complex urban street scene. The results are compared with those of several state-of-the-art models, including YOLO26n, YOLO-Master, IA-YOLO, and GDIP-YOLO.

First, in the extremely dark scene with a single bicycle, all evaluated methods successfully localize the target. However, notable differences are observed in detection confidence. Specifically, the confidence scores are 0.93 for YOLO26n, 0.94 for YOLO-Master, 0.79 for IA-YOLO, and 0.79 for GDIP-YOLO. In contrast, LUMA-YOLO achieves the highest confidence score of 0.95. This indicates that the proposed model effectively extracts discriminative object features even when illumination is severely limited.

Next, the performance is evaluated in a dimly lit staircase scenario involving two bicycles and one pedestrian. For the two bicycles, LUMA-YOLO obtains confidence scores

Table 4 Ablation study results on the NOD dataset with the random seed fixed to 0. The best results are indicated in **bold**

Model	LIAM	MMB	SimAM	mAP@0.5	mAP@0.5:0.95	Params (M)
Baseline				0.631	0.418	2.57
	✓			0.639	0.425	2.43
		✓		0.639	0.423	2.46
			✓	0.635	0.417	2.42
	✓	✓		0.649	0.432	2.47
		✓	✓	0.645	0.425	2.46
	✓		✓	0.645	0.428	2.43
LUMA-YOLO	✓	✓	✓	**0.657**	**0.434**	2.47

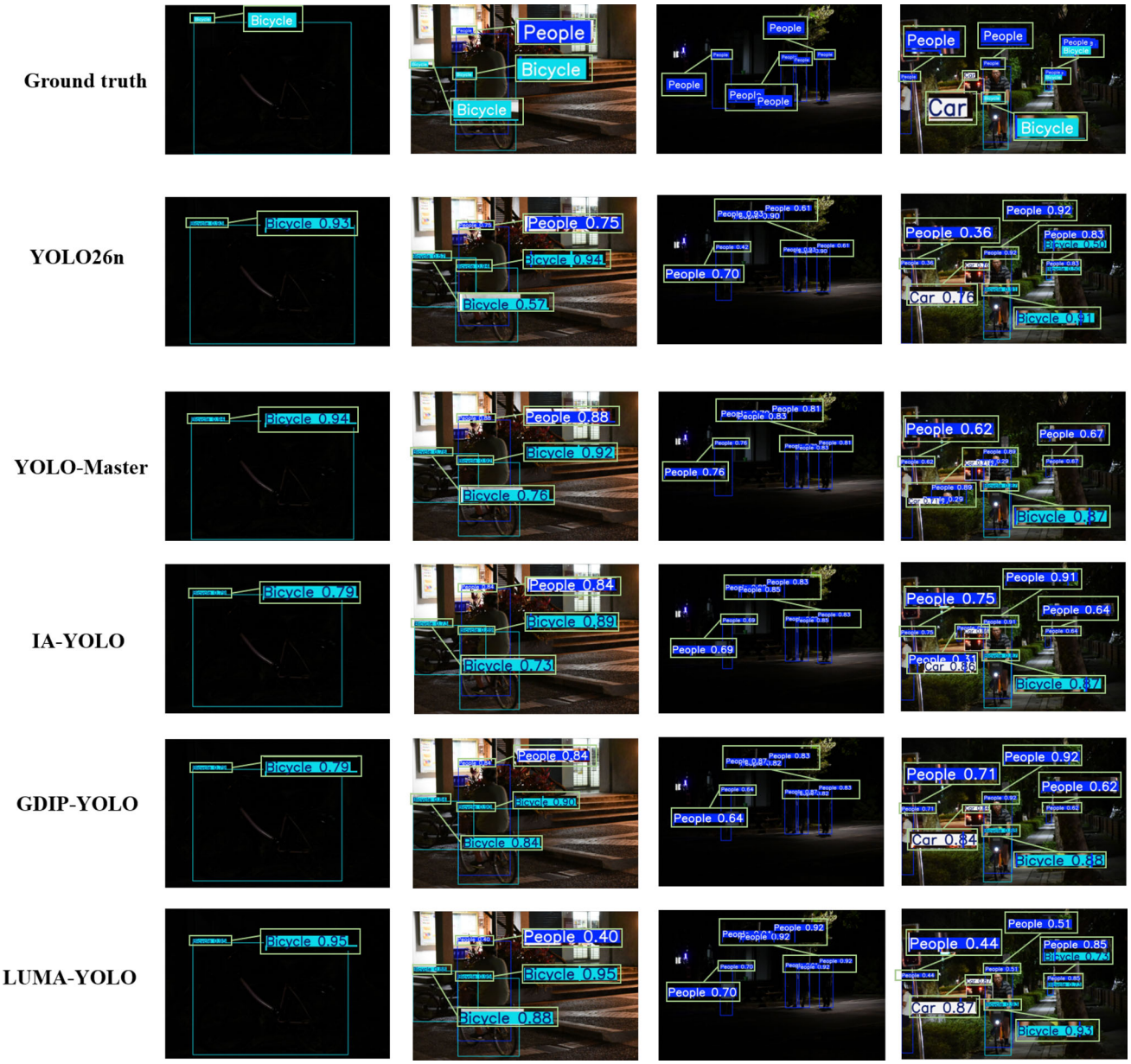


Fig. 11 Qualitative comparison of different methods in representative low-light scenarios on the NOD dataset. Rows from top to bottom correspond to the ground truth, YOLO26n, YOLO-Master, IA-YOLO, GDIP-YOLO, and LUMA-YOLO

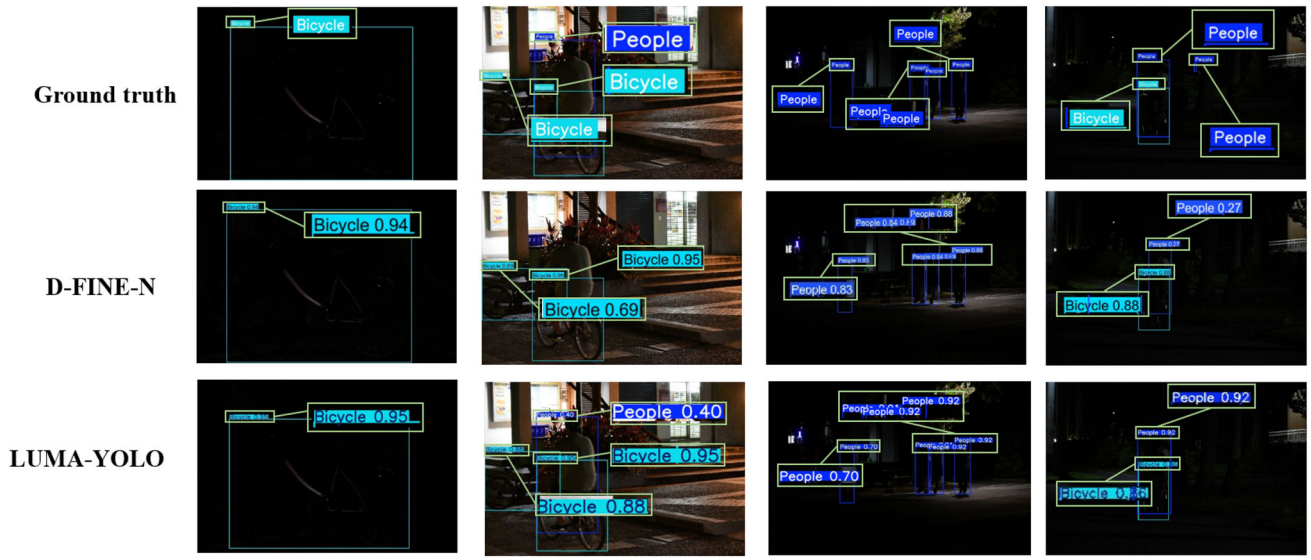


Fig. 12 Qualitative comparison between D-FINE-N and LUMA-YOLO on representative NOD low-light scenes. Rows from top to bottom correspond to the ground truth, D-FINE-N, and LUMA-YOLO

of 0.88 and 0.95, respectively. By comparison, YOLO26n gives scores of 0.57 and 0.94, YOLO-Master gives 0.76 and 0.92, IA-YOLO gives 0.73 and 0.89, and GDIP-YOLO gives 0.84 and 0.90. Although the pedestrian confidence of LUMA-YOLO is 0.40, lower than the 0.75-−0.88 range produced by the other models, it maintains superior localization and confidence for the primary foreground objects in the unevenly illuminated scene.

Subsequently, the methods are evaluated in a dark park scene containing multiple pedestrians. The confidence scores across the four primary targets reveal varying degrees of robustness among the models. YOLO26n yields scores of 0.70, 0.93, 0.90, and 0.61. YOLO-Master, IA-YOLO, and GDIP-YOLO achieve scores of 0.76, 0.70, 0.83, and 0.81; 0.69, 0.85, 0.80, and 0.83; and 0.64, 0.87, 0.82, and 0.83, respectively. LUMA-YOLO, by contrast, delivers consistently stable and high-confidence predictions in this scenario, with scores of 0.70, 0.91, 0.92, and 0.92. This stable performance underscores the model's capability to suppress background noise and reliably distinguish multiple targets.

Finally, the models are evaluated in a highly complex urban street scene comprising six targets: three pedestrians, one car, and two bicycles. In this challenging scenario, YOLO-Master, IA-YOLO, and GDIP-YOLO each miss one target entirely. While YOLO26n successfully detects all six targets, it yields lower confidence scores for some objects, with values of 0.36, 0.92, 0.83, 0.76, 0.50, and 0.91. LUMA-YOLO successfully localizes all six targets and provides confidence scores of 0.44, 0.51, 0.85, 0.87, 0.93, and 0.73. This demonstrates that LUMA-YOLO not only mitigates the missed detections observed in YOLO-Master, IA-YOLO, and GDIP-YOLO, but also maintains competitive confidence levels across dense targets, verifying its effectiveness in intricate, low-light road environments.

Figure 12 further compares LUMA-YOLO with D-FINE-N, a recent lightweight transformer-based detector, on the same NOD scenes. In the first column, both methods detect the bicycle in an almost completely dark image. The confidence scores are close, with 0.94 for D-FINE-N and 0.95 for LUMA-YOLO. This indicates that D-FINE-N can still localize the target when the object outline remains visible.

In the second column, D-FINE-N detects the two bicycles with confidence scores of 0.69 and 0.95, but it misses the pedestrian near the bright stair region. LUMA-YOLO detects the two bicycles with scores of 0.88 and 0.95, and it also keeps the pedestrian prediction with a confidence score of 0.40. Although this pedestrian score is modest, the target is not suppressed by the surrounding illumination, which reduces the risk of missed detection in uneven low-light scenes.

In the third column, both methods respond to the main pedestrians in the dark park scene. D-FINE-N gives high scores for several visible targets, such as 0.83 and 0.88. LUMA-YOLO also detects the pedestrian group, with confidence scores reaching 0.92 for the central and right-side targets, while retaining the low-contrast pedestrian on the left. The difference here is not a single confidence value, but the distribution of responses: LUMA-YOLO gives more consistent detections across both brighter and darker regions.

In the rightmost column, the scene becomes more difficult because the pedestrians and bicycle are located in a much darker area. D-FINE-N detects the bicycle with a confidence score of 0.88, but the pedestrian confidence drops to 0.27. LUMA-YOLO raises the visible pedestrian confidence to 0.92 and also detects the bicycle target.

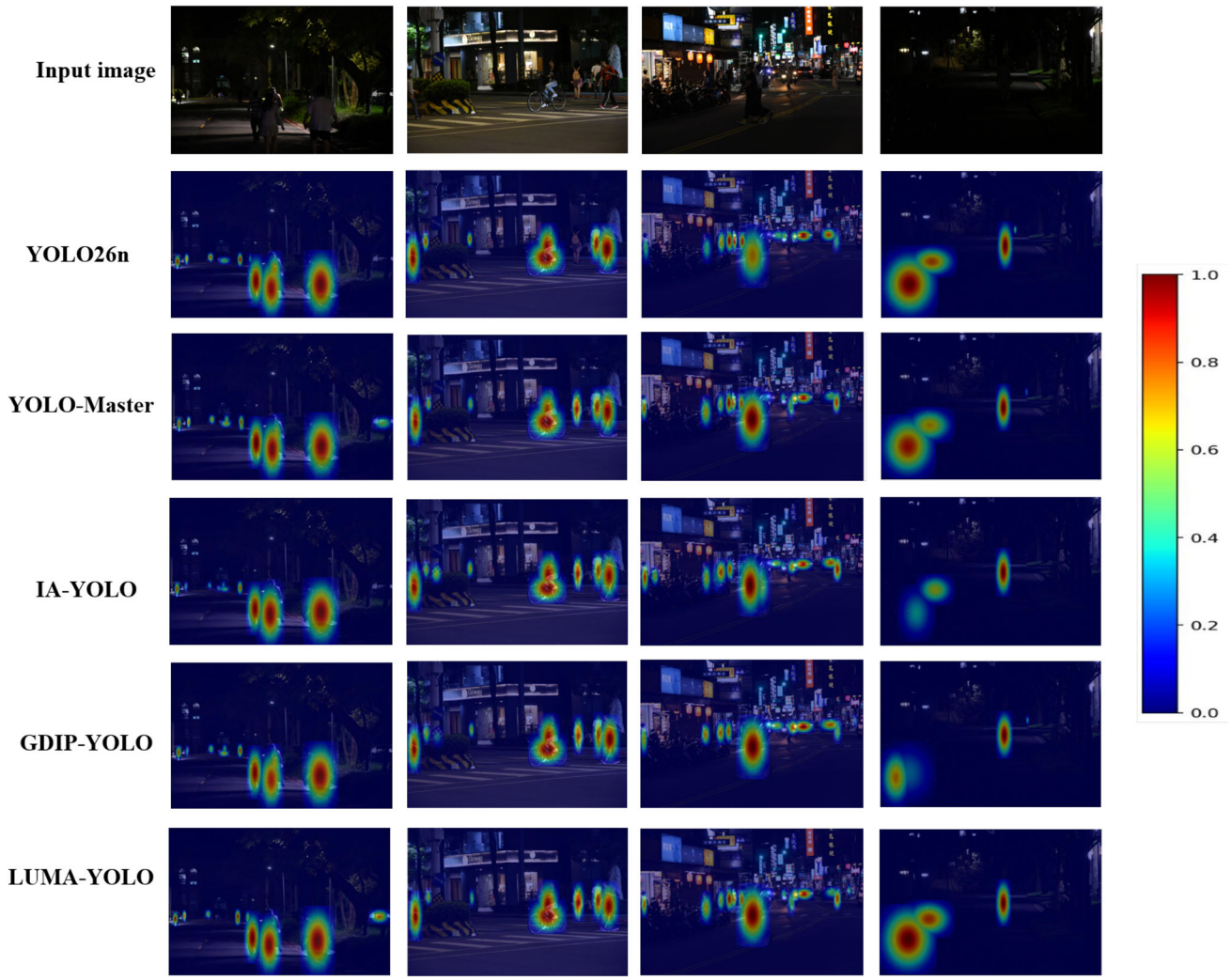


Fig. 13 Grad-CAM visualizations of the compared methods on representative low-light scenes from the NOD dataset. The top row shows the input images, and the remaining rows show the activation maps of YOLO26n, YOLO-Master, IA-YOLO, GDIP-YOLO, and LUMA-YOLO

As illustrated in Fig. 13, Grad-CAM is used to visualize the activation responses of LUMA-YOLO alongside YOLO26n, YOLO-Master, IA-YOLO, and GDIP-YOLO.

In the first and second columns, which correspond to a dimly lit park and an urban street crossing scene, YOLO26n, YOLO-Master, IA-YOLO, and GDIP-YOLO all show activations on the main pedestrians and bicycles to varying degrees. However, YOLO26n and YOLO-Master produce relatively scattered responses, with noticeable leakage into nearby background regions. IA-YOLO and GDIP-YOLO are somewhat more focused, but their responses remain less compact than those of LUMA-YOLO. By contrast, LUMA-YOLO concentrates its activations more tightly on the pedestrians and bicycles, suggesting better suppression of background interference under uneven illumination.

The third column depicts a crowded night street scene with multiple targets and background light interference. In

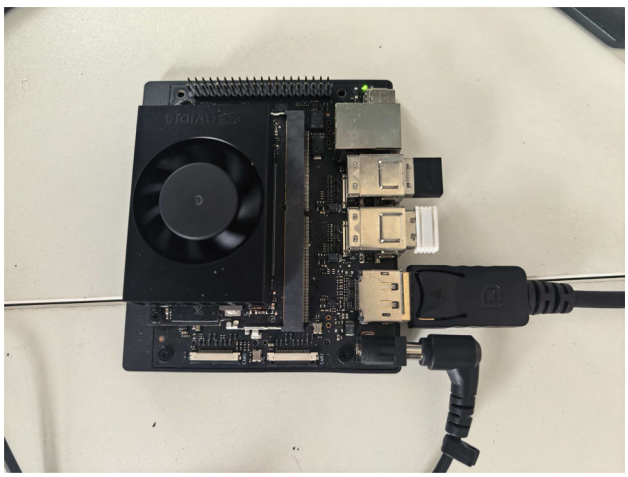


Fig. 14 NVIDIA Jetson Orin Nano Super 8 GB developer kit used for edge deployment evaluation



Fig. 15 On-device detection results of LUMA-YOLO on the Jetson Orin Nano Super 8 GB platform in two representative low-light road scenes. The left image shows a representative low-light street scene, and the right image shows another low-light road scene

this case, the differences among the methods become more apparent. The heatmaps of IA-YOLO and GDIP-YOLO are fragmented and diffuse, making it difficult for them to remain centered on the overlapping targets. YOLO26n and YOLO-Master achieve better localization, but their activations still drift toward the surrounding environment. LUMA-YOLO, in contrast, maintains concentrated responses on the visible targets, indicating that the proposed architecture suppresses surrounding light noise more effectively and preserves target boundaries in dense scenes.

In the fourth column, which represents an extremely low-light environment, the performance gap becomes more pronounced. Because the illumination is severely limited, the target features are heavily degraded. As a result, IA-YOLO and GDIP-YOLO produce weak or nearly vanished responses on the obscured target on the left. YOLO26n and YOLO-Master still identify the targets, but their activations remain less concentrated. LUMA-YOLO, however, continues to generate clear and strong responses aligned with the faint targets, suggesting a stronger ability to recover weak semantic features in extreme darkness and reduce missed detections.

Overall, the Grad-CAM visualizations provide intuitive evidence of the effectiveness of LUMA-YOLO. Compared with YOLO26n, YOLO-Master, IA-YOLO, and GDIP-YOLO, its heatmaps show more precise localization, stronger foreground responses, and less background leakage. These results are consistent with the quantitative improvements reported in the detection metrics and indicate that the proposed network handles complex illumination variations more effectively.

4.2.3 Edge Deployment Evaluation

To further assess practical deployability on embedded hardware, additional deployment experiments were conducted on an NVIDIA Jetson Orin Nano Super 8 GB developer kit. The board was configured in MAXN SUPER mode, and representative photographs of the hardware platform and on-device detection results are shown in Figs. 14 and 15. For consistency with the main comparison, five representative lightweight detectors, namely YOLOv11n [26], YOLOv12n [27], YOLOv13n [28], YOLO26n [29], and the proposed LUMA-YOLO, were evaluated on the NOD test set at an input resolution of 640×640 with a batch size of 1.

The Jetson deployment was performed in FP16 mode. The same preprocessing and evaluation protocol were used for all models, and several warm-up iterations were discarded before timing. The reported latency is the average single-image inference time measured on the test set, excluding disk I/O and result visualization. No TensorRT-specific engine optimization, INT8 calibration, or other post-training quantization was applied. Table 8 reports the corresponding accuracy and average inference time under this FP16 deployment setting.

As shown in Table 8, LUMA-YOLO maintains the best overall detection performance after deployment on the Jetson platform. FP16 reduces memory traffic and arithmetic cost on the Jetson GPU, which is reflected in the reported latency. Because the same precision setting was used for all models, the results in Table 8 are directly comparable under a common deployment condition. Without TensorRT-specific engine optimization or INT8 quantization, the reported num-

Table 8 Edge deployment results on the NOD dataset using FP16 inference on the NVIDIA Jetson Orin Nano Super 8 GB platform. The best and second-best results are indicated in **bold** and *italic*

Method	Precision	Recall	F1	mAP@0.5	mAP@0.5:0.95	Params (M)	Inference time (ms)
YOLOv11n [26]	0.778	0.542	0.639	0.628	0.400	2.60	34.18
YOLOv12n [27]	0.791	*0.549*	*0.648*	*0.643*	*0.420*	2.60	*33.14*
YOLOv13n [28]	*0.807*	0.540	0.647	0.637	0.419	*2.50*	51.26
YOLO26n [29]	**0.808**	0.527	0.638	0.631	0.418	2.57	**27.18**
LUMA-YOLO (Ours)	0.800	**0.556**	**0.656**	**0.657**	**0.434**	**2.47**	42.87

bers represent the FP16 accuracy-latency trade-off on the target device.

Although its precision of 0.800 is slightly lower than the best precision values achieved by YOLO26n and YOLOv13n, the proposed method obtains the highest recall, F1-score, mAP@0.5, and mAP@0.5:0.95 among all compared models, indicating that its predictions are more balanced and robust under embedded inference conditions. In particular, compared with YOLO26n, which provides the shortest latency of 27.18 ms, LUMA-YOLO improves recall from 0.527 to 0.556, F1-score from 0.638 to 0.656, mAP@0.5 from 0.631 to 0.657, and mAP@0.5:0.95 from 0.418 to 0.434. This behavior suggests that the proposed illumination-aware enhancement and lightweight feature-refinement strategy can preserve weak target cues more effectively even when computation is transferred from a desktop GPU to an embedded edge platform.

From the perspective of deployment efficiency, LUMA-YOLO runs at 42.87 ms per image on the Jetson Orin Nano Super 8 GB, corresponding to approximately 23.3 FPS, while retaining the smallest parameter count of only 2.47 M. Although this latency is not the lowest among all compared methods, it remains within a practical real-time range for low-light road-scene perception. More importantly, the additional latency relative to YOLO26n is accompanied by consistent gains in detection quality, rather than an accuracy drop. Compared with YOLOv13n, LUMA-YOLO also reduces the inference time from 51.26 ms to 42.87 ms while simultaneously improving recall, F1-score, mAP@0.5, and mAP@0.5:0.95. Therefore, the proposed model does not merely achieve compactness at the architectural level; it also demonstrates that the resulting accuracy-efficiency balance can be preserved after actual deployment on resource-constrained embedded hardware. These results provide further evidence that the proposed illumination-adaptive and lightweight design is suitable for practical low-light autonomous driving scenarios requiring both reliable detection and feasible edge inference speed.

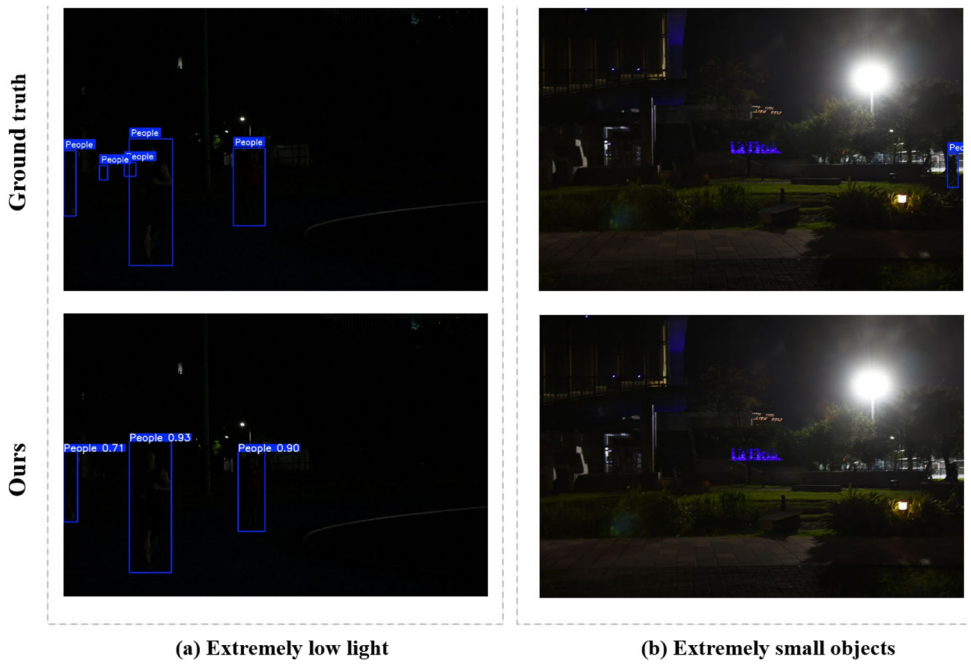


Fig. 16 Representative failure cases of LUMA-YOLO in challenging low-light scenes. The left example shows performance degradation under extremely weak illumination, where low-contrast pedestrians remain difficult to localize accurately. The right example shows a failure case caused by severe local glare, where strong overexposure suppresses surrounding target visibility and leads to missed detections

4.3 Failure-Case Analysis

Figure 16 presents two representative situations in which LUMA-YOLO still fails. In the left scene, several pedestrians appear in an area with extremely weak illumination, and their contours are barely separable from the background. Although the detector can still respond to some visible instances, the predicted boxes become unstable once the pedestrian shape is largely buried in darkness. This suggests that the current model still depends heavily on residual contour and contrast cues, and its localization degrades when these cues are severely weakened.

The right scene shows another limitation under strong local glare. The bright light source causes obvious saturation and compresses the contrast of nearby dark regions, making surrounding pedestrians less distinguishable. As a result, targets near the overexposed area are more likely to be missed. This indicates that the proposed enhancement and feature extraction strategy remains limited when strong illumination interference dominates the image.

These examples suggest that the remaining errors are mainly related to the coexistence of low visibility and severe local degradation, rather than low brightness alone. Similar difficulties are likely to arise in low-light scenes affected by rain, fog, or severe motion blur, where scattering or blur further weakens target boundaries and local contrast. Future work will, therefore, focus on improving robustness to glare, atmospheric scattering, and blur in challenging road environments.

5 Conclusion

This study proposed LUMA-YOLO, a lightweight illumination-adaptive object detector for low-light autonomous driving. To address the degradation caused by uneven illumination, LIAM was introduced at the input stage to estimate illumination and calibrate the image in a Retinex-inspired manner. To further improve feature extraction in low-light scenes, AConv was adopted to suppress noise while preserving local structure, MMB was embedded in the backbone to combine dynamic spatial cues with learned memory priors, and SimAM was inserted into the neck to strengthen multi-scale feature fusion without adding extra parameters.

Extensive experiments on the ExDark, NOD, and TDND datasets show that LUMA-YOLO consistently outperforms the compared methods in both accuracy and robustness. It achieves 0.709 mAP@0.5 and 0.451 mAP@0.5:0.95 on the ExDark dataset, 0.657 and 0.434 on the NOD dataset, and 0.725 and 0.461 on the TDND dataset. Meanwhile, the model remains compact, with only 2.47 M parameters and an inference speed of 57.8 FPS. The ablation study further confirms that each module contributes positively to the final performance, and the Grad-CAM visualizations indicate that LUMA-YOLO produces more concentrated target responses with less background leakage in challenging low-light scenes.

Overall, LUMA-YOLO provides an effective balance between detection accuracy and computational cost, making it suitable for real-time low-light object detection in autonomous driving. For practical use, the results suggest several recommendations. First, in low-light road perception systems, illumination adaptation should be optimized together with the detector rather than used only as an independent visual enhancement step. Second, for edge deployment, model selection should consider recall, localization accuracy, and latency together, since a detector selected only for minimum latency may miss weakly illuminated pedestrians or bicycles. Third, when deploying LUMA-YOLO in real vehicles or roadside devices, additional validation is still recommended for scenes dominated by strong glare, rain, fog, or severe motion blur, because these conditions can further weaken object boundaries. In such cases, the detector should be combined with scene-specific data collection or complementary sensing when available.

Future work will focus on improving robustness under heavy occlusion, motion blur, and adverse weather conditions, and on extending the framework to more complex road scenarios.

Author Contributions All authors contributed to the study conception, method design, experiments, and manuscript preparation.

Funding This research received no external funding.

Data Availability The datasets used in this study are publicly available (ExDark, NOD, and TDND).

Code Availability The code is available at https://github.com/Softnameli/YOLO.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Ren, S.; He, K.; Girshick, R.; Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. IEEE Trans. Pattern Anal. Mach. Intell. **39**(6), 1137–1149 (2017). https://doi.org/10.1109/TPAMI.2016.2577031
2. Liu, W.; Anguelov, D.; Erhan, D.; Szegedy, C.; Reed, S.; Fu, C..-Y.; Berg, A.C.: European conference on computer vision (ECCV), p. 21–37 (2016). https://doi.org/10.1007/978-3-319-46448-0_2
3. Redmon J.; Farhadi A.: YOLOv3: An incremental improvement. arXiv preprint arXiv:1804.02767 (2018)https://doi.org/10.48550/arXiv.1804.02767
4. Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; Zagoruyko, S.: End-to-end object detection with transformers. In:

European Conference on Computer Vision (ECCV), pp. 213–229 (2020). https://doi.org/10.1007/978-3-030-58452-8_13
5. Cai, Y.; Bian, H.; Lin, J.; Wang, H.; Timofte, R.; Zhang, Y.: Retinexformer: One-stage Retinex-based transformer for low-light image enhancement. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 12504–12513 (2023). https://doi.org/10.1109/ICCV51070.2023.01149
6. Wu, Y.; Pan, C.; Wang, G.; Yang, Y.; Wei, J.; Li, C.; Shen, H.T.: Learning semantic-aware knowledge guidance for low-light image enhancement. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1662–1671 (2023). https://doi.org/10.1109/CVPR52729.2023.00166
7. He, Z.; Ran, W.; Liu, S.; Li, K.; Lu, J.; Xie, C.; Liu, Y.; Lu, H.: Low-light image enhancement with multi-scale attention and frequency-domain optimization. IEEE Trans. Circuits Syst. Video Technol. **34**(4), 2861–2875 (2024). https://doi.org/10.1109/TCSVT.2023.3313348
8. Du, Z.; Shi, M.; Deng, J.: Boosting object detection with zero-shot day-night domain adaptation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12666–12676(2024). https://doi.org/10.1109/CVPR52733.2024.01204
9. Wang, W.; Luo, R.; Yang, W.; Liu, J.: Unsupervised illumination adaptation for low-light vision. IEEE Trans. Pattern Anal. Mach. Intell. **46**(9), 5951–5966 (2024). https://doi.org/10.1109/TPAMI.2024.3382108
10. Hui, Y.; Wang, J.; Li, B.: WSA-YOLO: weak-supervised and adaptive object detection in the low-light environment for YOLOV7. IEEE Trans. Instrumentation Measurement **73**, 1–12 (2024). https://doi.org/10.1109/TIM.2024.3350120
11. Wu, H.; Zhang, R.; Chen, J.; Lyu, N.; Hu, H.; Guo, Y.; Qiu, T.Z.: WTEFNet: real-time low-light object detection for advanced driver assistance systems. IEEE Trans. Instrum. Measurement **75**, 1–17 (2026). https://doi.org/10.1109/TIM.2026.3655940
12. Yan Q.; Feng Y.; Zhang C.; Pang G.; Shi K.; Wu P.; Dong W.; Sun J.; Zhang Y.: HVI: a new color space for low-light image enhancement. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5678–5687 (2025). https://doi.org/10.1109/CVPR52734.2025.00533
13. Zhou H.; Dong W.; Liu X.; Liu S.; Min X.; Zhai G.; Chen J.: GLARE: low light image enhancement via generative latent feature based codebook retrieval. In: European Conference on Computer Vision (ECCV), pp. 36–54 (2024). https://doi.org/10.1007/978-3-031-73195-2_3
14. Fu, Z.; Yang, Y.; Tu, X.; Huang, Y.; Ding, X.; Ma, K..-K.: Learning a simple low-light image enhancer from paired low-light instances. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 22252–22261 (2023). https://doi.org/10.1109/CVPR52729.2023.02131
15. Tang, W.H.; Yuan, H.; Chiang, T..-H.; Huang, C..-C.: Zero-LEINR: zero-reference low-light image enhancement with intrinsic noise reduction. In: 2023 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5 (2023). https://doi.org/10.1109/ISCAS46773.2023.10181743
16. Zhu X.; Su W.; Lu L.; Li B.; Wang X.; Dai J.: Deformable DETR: deformable transformers for end-to-end object detection. In: International Conference on Learning Representations (ICLR) (2021). https://openreview.net/forum?id=gZ9hCDWe6ke
17. Meng, D.; Chen, X.; Fan, Z.; Zeng, G.; Li, H.; Yuan, Y.; Sun, L.; Wang, J.: Conditional DETR for fast training convergence. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 3651–3660 (2021). https://doi.org/10.1109/ICCV48922.2021.00363
18. Wang Y.; Zhang X.; Yang T.; Sun J.: Anchor DETR: Query design for transformer-based detector. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36(3), pp. 2567–2575 (2022). https://doi.org/10.1609/aaai.v36i3.20158
19. Liu S.; Li F.; Zhang H.; Yang X.; Qi X.; Su H.; Zhu J.; Zhang L.: DAB-DETR: dynamic anchor boxes are better queries for DETR. In: International Conference on Learning Representations (ICLR) (2022). https://openreview.net/forum?id=oMI9PjOb9Jl
20. Li F.; Zhang H.; Liu S.; Guo J.; Ni L. M.; Zhang L.: DN-DETR: Accelerate DETR training by introducing query DeNoising. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),pp. 13619–13627 (2022). https://doi.org/10.1109/CVPR52688.2022.01325
21. Zhang H.; Li F.; Liu S.; Zhang L.; Su H.; Zhu J.; Ni L. M.; Shum H.-Y.: DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In: International Conference on Learning Representations (ICLR) (2023). https://openreview.net/forum?id=3mRwyG5one
22. Zhao, Y.; Lv, W.; Xu, S.; Wei, J.; Wang, G.; Dang, Q.; Liu, Y.; Chen, J.: DETRs Beat YOLOs on real-time object detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 16965–16974 (2024). https://doi.org/10.1109/CVPR52733.2024.01605
23. Lv W.; Zhao Y.; Chang Q.; Huang K.; Wang G.; Liu Y.: RT-DETRv2: Improved baseline with bag-of-freebies for real-time detection transformer (2024). arXiv preprint arXiv:2407.17140. https://doi.org/10.48550/arXiv.2407.17140
24. Chen Q.; Su X.; Zhang X.; Wang J.; Chen J.; Shen Y.; Han C.; Chen Z.; Xu W.; Li F.; Zhang S.; Yao K.; Ding E.; Zhang G.; Wang J.: LW-DETR: a transformer replacement to YOLO for real-time detection (2024). arXiv preprint arXiv:2406.03459. https://doi.org/10.48550/arXiv.2406.03459
25. Peng Y.; Li H.; Wu P.; Zhang Y.; Sun X.; Wu F.: D-FINE: redefine regression task in DETRs as fine-grained distribution refinement. In: International Conference on Learning Representations (ICLR) (2025). https://openreview.net/forum?id=MFZjrTFE7h
26. Jocher G.; Qiu J.: Ultralytics YOLO11. Computer software, version 11.0.0 (2024). https://docs.ultralytics.com/models/yolo11. Accessed 3 June 2026.
27. Tian Y.; Ye Q.; Doermann D.: YOLOv12: attention-centric real-time object detectors. In: Advances in Neural Information Processing Systems 38 (NeurIPS), pp. 78433–78457 (2025). https://doi.org/10.52202/085713-2627
28. Lei M.; Li S.; Wu Y.; Hu H.; Zhou Y.; Zheng X.; Ding G.; Du S.; Wu Z.; Gao Y.: YOLOv13: Real-time object detection with hypergraph-enhanced adaptive visual perception (2025). arXiv preprint arXiv:2506.17733. https://doi.org/10.48550/arXiv.2506.17733
29. Jocher G.; Qiu J.; Liu M.; Lyu S.; Akyon F. C.; Kalfaoglu M. E.: Ultralytics YOLO26: Unified real-time end-to-end vision models (2026). arXiv preprint arXiv:2606.03748. https://doi.org/10.48550/arXiv.2606.03748
30. Zhang B.; Guo Y.; Yang R.; Zhang Z.; Xie J.; Suo J.: DarkVision: a benchmark and study for low-light image/video analysis. Comput. Visual Media, **12**(3), 615–642 (2026). https://doi.org/10.26599/CVM.2025.9450461
31. Xue, R.; Duan, J.; Du, Z.: MPE-DETR: a multiscale pyramid enhancement network for object detection in low-light images. Image Vision Comput. **150**, 105202 (2024). https://doi.org/10.1016/j.imavis.2024.105202
32. Hashmi, K.A.; Kallempudi, G.; Stricker, D.; Afzal, M.Z.: FeatEnHancer: enhancing hierarchical features for object detection and beyond under low-light vision. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 6725–6735 (2023). https://doi.org/10.1109/ICCV51070.2023.00619
33. Hong M.; Cheng S.; Huang H.; Fan H.; Liu S.: You only look around: learning illumination-invariant feature for low-light object detection. In: Advances in Neural Information Processing Systems

37 (NeurIPS), pp. 87136–87158 (2024). https://doi.org/10.52202/079017-2765
34. Peng, D.; Ding, W.; Zhen, T.: A novel low light object detection method based on the YOLOv5 fusion feature enhancement. Scientifi Rep. **14**, 4486 (2024). https://doi.org/10.1038/s41598-024-54428-8
35. Wang, X.; Yang, R.; Wu, Z.; Sun, L.; Liu, J.; Zou, L.: ILENet: illumination-modulated Laplacian-Pyramid enhancement network for low-light object detection. Expert Syst. Appl. **271**, 126504 (2025). https://doi.org/10.1016/j.eswa.2025.126504
36. Liu W.; Ren G.; Yu R.; Guo S.; Zhu J.; Zhang L.: Image-adaptive YOLO for object detection in adverse weather conditions. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36(2), pp. 1792–1800 (2022). https://doi.org/10.1609/aaai.v36i2.20072
37. Kalwar, S.; Patel, D.; Aanegola, A.; Konda, K.R.; Garg, S.; Krishna, K.M.: GDIP: gated differentiable image processing for object detection in adverse conditions, p. 7083–7089 (2023). https://doi.org/10.1109/ICRA48891.2023.10160356
38. Ma L.; Ma T.; Liu R.; Fan X.; Luo Z.: Toward fast, flexible, and robust low-light image enhancement. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 5637–5646(2022). https://doi.org/10.1109/CVPR52688.2022.00555
39. Wang, Q.; Wu, B.; Zhu, P.; Li, P.; Zuo, W.; Hu, Q.: ECA-Net: efficient channel attention for deep convolutional neural networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 11534–11542 (2020). https://doi.org/10.1109/CVPR42600.2020.01155
40. Hu, J.; Shen, L.; Sun, G.: Squeeze-and-excitation networks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7132–7141 (2018). https://doi.org/10.1109/CVPR.2018.00745
41. Woo, S.; Park, J.; Lee, J..-Y.; Kweon, I.S.: European conference on computer vision (ECCV), p. 3–19. (2018). https://doi.org/10.1007/978-3-030-01234-2_1
42. Yang L.; Zhang R.-Y.; Li L.; Xie X.: SimAM: a simple, parameter-free attention module for convolutional neural networks. In: Proceedings of the 38th International Conference on Machine Learning (ICML), PMLR, vol. 139, pp. 11863–11874 (2021). https://proceedings.mlr.press/v139/yang21o.html
43. Loh, Y.P.; Chan, C.S.: Getting to know low-light images with the exclusively dark dataset. Comput. Vision Image Underst. **178**, 30–42 (2019). https://doi.org/10.1016/j.cviu.2018.10.010
44. Morawski, I.; Chen, Y..-A.; Lin, Y..-S.; Hsu, W.H.: NOD: Taking a closer look at detection under extreme low-light conditions with night object detection dataset. In: British Machine Vision Conference (BMVC), pp. 1–13 (2021). https://doi.org/10.5244/C.35.317
45. Nie, C.; Qadar, M.A.; Zhou, S.; Zhang, H.; Shi, Y.; Gao, J.; Sun, Z.: Transnational image object detection datasets from nighttime driving. Signal Image Video Process. **17**(4), 1123–1131 (2023). https://doi.org/10.1007/s11760-022-02319-8
46. Jocher G.; Chaurasia A.; Qiu J.: Ultralytics YOLOv8. Computer software, version 8.0.0 (2023). https://docs.ultralytics.com/models/yolov8. Accessed 3 June 2026.
47. Wang A.; Chen H.; Liu L.; Chen K.; Lin Z.; Han J.; Ding G.: YOLOv10: real-time end-to-end object detection. In: Advances in Neural Information Processing Systems 37 (NeurIPS), pp. 107984–108011 (2024). https://doi.org/10.52202/079017-3429
48. Lin X.; Peng J.; Gan Z.; Zhu J.; Liu J.: YOLO-Master: MOE-accelerated with specialized transformers for enhanced real-time detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 18440–18449 (2026). https://doi.org/10.48550/arXiv.2512.23273

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.